

AMARIS: A Memory-Augmented Rubric Improvement System for Rubric-Based Reinforcement Learning

Peilin Wu¹, Xinlu Zhang³, Kun Wan², Wentian Zhao², Gang Wu², Xinya Du¹, Zhiyu Chen¹

¹The University of Texas at Dallas, ²Adobe Inc.

³Department of Computer Science, University of California, Santa Barbara,
{peilin.wu, zhiyu.chen2}@utdallas.edu

Abstract

Rubric-based reward shaping provides interpretable and editable reward signals for fine-tuning LLMs via reinforcement learning (RL), but existing adaptive rubric methods typically update criteria from local evidence such as the current batch or instance-level comparisons. This local view discards diagnostic information produced during training, making it difficult to track recurring failures, evaluate previous rubric edits, or raise standards once earlier criteria become saturated. We introduce AMARIS, A Memory-Augmented Rubric Improvement System that grounds rubric updates in longitudinal training evidence. AMARIS stores rollout analyses, step-level summaries, and rubric update records in a persistent evaluation memory, then retrieves recent and semantically relevant history to revise rubrics. We evaluate AMARIS across science, medicine, instruction following, and creative writing under both global and instance-specific rubric settings. AMARIS improves over static, local-adaptive, and memory-ablated baselines, such as +2.8 points on GPQA-Diamond and +2.2 points on IFBench over the strongest baselines, while analysis shows that memory reduces oscillatory rubric edits and supports a progression from early failure correction to later curriculum advancement. AMARIS runs asynchronously alongside the normal RL loop, reducing blocking latency relative to synchronous rubric updates. The source code of AMARIS is at <https://github.com/qualideal217/AMARIS>.

instruction following, making the reward function interpretable and editable in open-ended settings where a single ground-truth verifier is hard to define (Zheng et al., 2023). Rubrics also allow practitioners to inspect which behaviors are being rewarded and revise the reward specification when training exposes new failure modes, including shortcuts that raise reward without improving output quality (Skalse et al., 2022; Gao et al., 2023). However, a fixed rubric set defined before training cannot anticipate how policy behavior changes during RL training, motivating adaptive rubric updates.

Recent work improves rubric flexibility in several ways. Some methods generate criteria for each input (Gupta et al., 2025) or let the policy model propose rubrics for its own outputs (Sheng et al., 2026; Xu et al., 2026). Others update rubrics from pairwise comparisons between current and reference rollouts (Rezaei et al., 2025), joint optimization with the judge (Xu et al., 2026), or batch-level selection of criteria that best distinguish current rollouts (Shao et al., 2025). These approaches show that rubrics can be adapted rather than fixed before training. However, most adaptive rubric updates rely on local signals such as the current instance or immediate comparisons between instances, while diagnostic evidence from earlier training steps is discarded after immediate use. As a result, each update has limited access to how policy behavior has changed over time or how previous rubric edits affected the reward signal. Reusing this evidence can make rubric adaptation more consistent across training steps and can support updates that reflect both current rollout quality and earlier evaluation history (Li et al., 2025; Freitag et al., 2025).

We introduce AMARIS, A Memory-Augmented Rubric Improvement System for adaptive rubric-based reinforcement learning. AMARIS stores rollout-level diagnostic reports, step-level summaries, and rubric update records in a persistent evaluation memory. At each RL update step, it

1 Introduction

Rubric-based reward shaping has become a practical mechanism for training language models in domains where output quality is multi-dimensional. Rubric-based approaches construct reward signals by decomposing evaluation into explicit natural-language criteria with associated weights (Kim et al., 2024; Ye et al., 2025). These criteria can cover correctness, helpfulness, safety, style, and

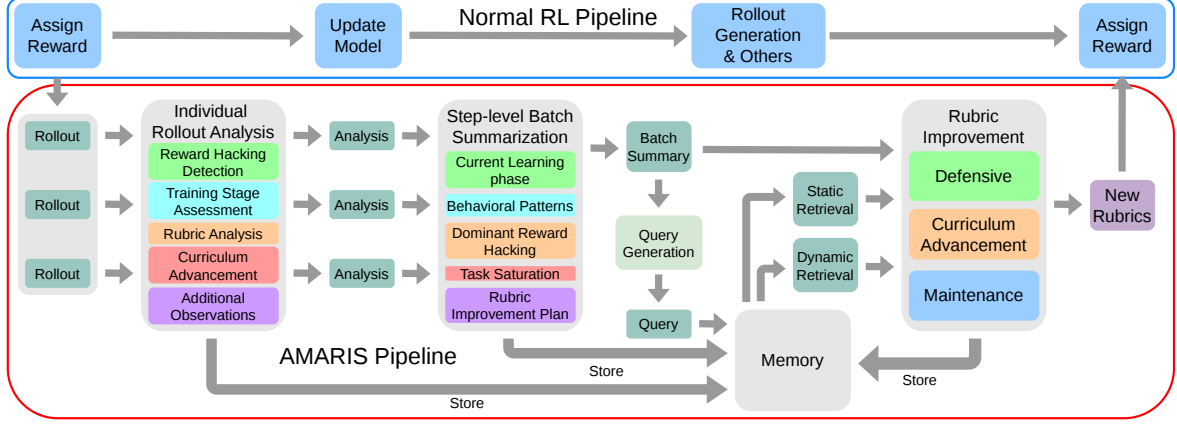


Figure 1: The AMARIS system operates asynchronously in parallel with the normal RL pipeline to continuously refine rubrics.

retrieves both recent summaries and semantically relevant historical evidence, then updates the active rubric set through defensive correction, curriculum advancement, or maintenance. This design turns rubric adaptation from a local per-step heuristic into a longitudinal evaluation process grounded in accumulated training evidence.

We evaluate AMARIS across science, medicine, instruction following, and creative writing under both global and per-instance rubric settings. AMARIS improves over static, local-adaptive, and memory-ablated baselines, including +2.8 points on GPQA-Diamond, +3.0 points on HealthBench, and +2.2 points on IFBench over the strongest non-AMARIS baselines. Ablations show that recent-step retrieval and semantic retrieval provide complementary gains, while latency analysis shows that asynchronous execution substantially reduces wall-clock delay relative to synchronous rubric updates. Analyses of rubric evolution and qualitative cases further show how memory supports defensive correction, curriculum advancement, and late-stage maintenance.

Our contributions are as follows:

- We identify persistent evaluation memory as a useful state variable for adaptive rubric-based reward shaping.
- We propose AMARIS, a memory-augmented rubric improvement system that stores rollout analyses, step-level summaries, and rubric update records, and retrieves both recent and semantically matched history to guide rubric updates.

- We evaluate AMARIS under global and per-instance rubric settings across four domains and analyze rubric evolution, stability, and pipeline latency, showing how memory supports defensive correction, curriculum advancement, and maintenance.

2 AMARIS

AMARIS introduces a memory-augmented rubric improvement loop into rubric-based reinforcement learning. The procedure has three stages. First, AMARIS analyzes individual rollouts as described in Section 2.1. Second, it summarizes rollout analyses into a step-level batch summary as described in Section 2.2. Third, it improves rubrics by grounding every change in recent and semantically matched analyses retrieved from memory as described in Section 2.3. Section 2.4 describes how this procedure can be executed asynchronously and in parallel with the normal RL pipeline.

2.1 Individual Rollout Analysis

In a standard rubric-based RL paradigm, the policy model’s rollouts $y \sim \pi_\phi(\cdot \mid x)$, where π_ϕ is the policy model, x is the input, and y is the output, are evaluated against a set of active rubrics. At each RL training step t , AMARIS maintains an active rubric set $G_t = \{(g_j, w_j)\}_{j=1}^{n_{g,t}}$ where g_j is the natural language definition of each individual rubric, w_j is the scalar weight, and $n_{g,t}$ is the number of active rubrics. This rubric set can be global, with one set and one evaluation memory shared across all inputs, or per-instance, with each unique input maintaining its own set and memory. We use G_t for both rubric

scopes in the main text. In the per-instance setting, it denotes the active rubric set associated with the current input. The final scalar reward r_t is then generated as $r_t(x, y) = S_\theta(x, y, G_t)$, where S_θ is the process of scoring using an LLM as a judge (prompt template in Figure 3-4). The scalar reward is the scoring signal used by RL. Since scalar rewards may hide model behaviors such as reward hacking and rubric limitations, AMARIS also performs a deeper diagnosis of each individual rollout about the model’s behavior and how the current rubrics shape it. The later rubric update stage uses batch summaries and retrieved memory to produce the next active rubric set.

For each rollout, AMARIS receives the model’s input and output, the current rubric set, training metadata (the high-level goal, current step, total steps, model size, etc.), and supplemental context such as ground-truth answers, denoted by z , and produces a structured diagnostic report $a = A_\psi(x, y, G_t, z)$, where A_ψ is the LLM for analysis (prompt template in Figures 5-8). Each analysis provides five types of information. (i) **Reward hacking detection** reports whether the rollout shows signs of reward hacking, such as over-refusal for safety or sycophancy, with evidence and confidence level. (ii) **Training stage assessment** judges whether the observed performance is expected given the model size at the current step. (iii) **Rubric analysis** explains how the current rubrics may have shaped the observed behavior and whether there are specific flaws in rubrics or weights that incentivize undesired outputs. (iv) **Curriculum advancement** identifies which capability or behavior should be prioritized next to advance the model toward the high-level training goal. (v) **Additional observations** preserve other noteworthy signals that may be relevant for later analysis. The resulting structured analysis is stored in the persistent evaluation memory, preserving the evidence needed for step-level summarization in Section 2.2 and rubric improvement in Section 2.3.

2.2 Step-Level Batch Summarization

Individual rollout analyses do not capture the overall step-level behavior and are too local to justify reliable rubric changes on their own. AMARIS therefore aggregates the individual analyses collected for the update at step t into a step-level summary about what the model is currently doing, where it is failing, and whether the current rubrics remain

informative.

Once all the analyses $\{a_j\}_{j=1}^{n_{a,t}}$ for individual rollouts are produced, where $n_{a,t}$ is the number of analyzed rollouts for the update at step t , AMARIS summarizes them into a step-level batch summary $b_t = B_\psi(\{a_j\}_{j=1}^{n_{a,t}})$, where B_ψ is the LLM for batch summarization (prompt template in Figures 9-12). The summary contains the general health of training and the current learning phase, the behavioral patterns that recur throughout the batch, the dominant reward hacking risk and its evidence, the weakest rubric, signs of task saturation and a provisional plan for the rubric improvement. By synthesizing step-level trends, b_t provides the rubric improvement stage with a view of what the policy is currently doing and where the current set of rubrics fall short.

When an update contains more analyses than can be processed in a single summarization pass, AMARIS summarizes smaller chunks independently and consolidates the intermediate summaries into the same final step-level summary format. This hierarchical strategy, referred to as meta summarization, allows the pipeline to scale to large rollout batches while preserving a fixed downstream interface for the retrieval and rubric improvement stages. The step-level batch summaries are also stored in the persistent evaluation memory for the future rubric improvement stage in Section 2.3.

2.3 Memory-Augmented Rubric Improvement

Without memory, rubric updates are reactive and can respond only to the changes for the current step, with no mechanism for recalling whether a similar suboptimal behavior appeared earlier, a previous rubric update failed, or the model has been steadily improving. AMARIS addresses this limitation through persistent memory together with a hybrid retrieval strategy.

Persistent evaluation memory. AMARIS stores all previous evaluations in a database that persists throughout training as the memory available at step t , denoted by M_t . The memory continuously accumulates individual rollout analyses a , step-level summaries b , chunk-level summaries if meta summarization is enabled, and rubric improvement records u . Here, u is a structured record containing the rubric update strategy, retrieved history consulted, rubric edit operations, resulting adaptive rubric set, and a short summary for future steps.

Each evaluation document is indexed for later retrieval. The memory therefore contains rich information about the past evaluations and supports various retrieval strategies.

Static and dynamic retrieval. Before each update, AMARIS retrieves relevant historical context I_t from static and dynamic retrieval. Static retrieval supplies recent step-level summaries, giving the updater a compact view of the recent training trajectory with temporal continuity. Dynamic retrieval reads the current step-level summary and uses the query generator Q_ψ to generate targeted search queries (prompt template in Figure 17-19). These queries usually cover the dominant failure mode, suspected rubric flaw, possible curriculum objective, and failure from similar past updates. The retrieved documents may include earlier individual analyses, summaries, and rubric update records from any point in training. Duplicate documents are removed before rubric update. This retrieval policy keeps the updater context bounded while maintaining flexibility.

Rubric improvement. Based on the retrieval results, AMARIS generates the next set of rubrics as $G_{t+1}, u_t = U_\psi(G_t, b_t, I_t)$, where U_ψ is the LLM for rubric improvement (prompt template in Figures 20-23). AMARIS selects among three broad strategies. A **Defensive** update modifies rubrics and weights to reduce suboptimal behaviors and uses retrieved memory to check whether the same suboptimal behavior has appeared before. A **Curriculum advancement** update raises the evaluation standard when the current rubrics appear saturated and retrieved memory confirms that the model has mastered the easier requirements. A **Maintenance** update keeps the rubric set stable when the current evidence supports stability. To realize its chosen strategy, the LLM for rubric update U_ψ may apply any combination of rubric operations. **Create** adds new rubrics, **Update** revises rubric text for precision, **Delete** removes useless or counterproductive rubrics, **Reweight** shifts training emphasis, **Merge** consolidates overlapping rubrics, and **Split** decomposes one rubric into atomic rubrics. U_ψ also produces reasoning summarizing what history was consulted and what lessons should be retained. The resulting update record is written back to the memory so that every update becomes available as evidence for subsequent updates.

2.4 RL Pipeline Integration

The rubric improvement procedure would add substantial blocking latency if executed synchronously within each RL step. To avoid blocking the RL training loop, AMARIS executes the procedure asynchronously and in parallel with the normal RL pipeline. Specifically, for every step, rollouts are scored immediately using the currently active set of rubrics, while the rubric improvement proceeds in the background. Once the update completes, the updated rubrics become active for subsequent scoring. Although such asynchronous execution causes the rubrics to be slightly off-policy, it reduces blocking latency by moving rubric improvement off the critical path. It also requires less interference with the original RL training process and fewer changes to the code of the RL training pipeline.

3 Experiment Setup

This section describes the experimental framework used to evaluate AMARIS. We outline the datasets, evaluation benchmarks, baselines, and implementation details to provide a clear context for our results.

3.1 Datasets & Evaluation

We evaluate AMARIS across four diverse domains with both open-ended and closed-ended tasks. For each domain, we pair a rubric-annotated training set with one or more evaluation benchmarks. (i) **Science.** We train on RaR-Science (Gunjal et al., 2025), a corpus of scientific questions annotated with rubrics, and evaluate on GPQA-Diamond (Rein et al., 2024). We report average accuracy (%) over 10 measurements as the metric. (ii) **Medicine.** We train on RaR-Medicine (Gunjal et al., 2025), a medical variant of the rubric-annotated training data, and evaluate on HealthBench (Arora et al., 2025). We report the HealthBench composite score as the metric. (iii) **Instruction following.** We follow the experimental setup of OpenRubrics (Liu et al., 2026) and train on the OpenRubrics (Liu et al., 2026) dataset. We evaluate on three instruction-following benchmarks: IFEval (Zhou et al., 2023), InfoBench (Qin et al., 2024), and IFBench (Pyatkin et al., 2025), which we report the overall accuracies of each of them. (iv) **Creative writing.** We train on the writing subset of RubricHub (Li et al., 2026), a large-scale rubric-annotated dataset. We evaluate on WritingBench (Wu et al., 2025b) and Creative Writing v3 (Paech,

2025) from EQ-Bench and report the overall scores as the metric, also referred as CW-v3 in the following sections.

3.2 Baselines

We compare AMARIS against several baselines. Three baselines apply across all four domains: (1) Naive (no RL): the base model without any RL fine-tuning. (2) RubricHub (Li et al., 2026): a large scale rubric-annotated dataset with evaluations on rubric-based reinforcement learning (RuRL) that covers all the domains we tested. (3) RuscaRL (Zhou et al., 2026): a rubric-based RL method that we evaluate across all four domains. We also include domain-specific baselines. (i) **Science & Medicine.** We include RaR (Gunjal et al., 2025), which uses a fixed set of rubrics throughout training on the same RaR-Science and RaR-Medicine data. (ii) **Instruction following.** We include OpenRubrics (Liu et al., 2026), Rubric-ARM (Xu et al., 2026), and RLCF (Viswanathan et al., 2025). For all the baselines, we report the values obtained by re-training and re-evaluating using the same base model as ours, if possible. If not, we reference their reported values.

3.3 Implementation Details

Unless otherwise stated, we perform RL training with Qwen2.5-7B-Instruct (Qwen et al., 2025) as the base model, GRPO (Shao et al., 2024) as the RL algorithm with group-wise and batch-wise reward normalization enabled, using 4 NVIDIA A100 80GB GPUs. We train all models with up to 400 steps and save the checkpoint every 50 steps, and use the final saved checkpoint for testing to ensure a fair evaluation. By default, we use GPT-OSS-120B (OpenAI et al., 2025) for scoring and individual analysis, and GPT-5.1 (Singh et al., 2025) for the remaining usages. This allocation reflects that scoring and individual analysis account for the majority of token consumption and therefore benefit most from a cost-effective model, while the remaining stages consume fewer tokens but demand higher reasoning capability. We analyze the influence of LLM choices in Appendix D, and test an alternative Qwen3-4B policy base in Appendix F.3. The memory is implemented with Chroma (chroma-core, 2026) as the vector database. For reward computation, following previous works (Gunjal et al., 2025; Li et al., 2026), we use a single LLM call per rollout. For all the LLM usages including the policy and AMARIS, we use the default setting for

inferences.

Unless otherwise stated, we evaluate both rubric scopes introduced in Section 2. In the global setting, AMARIS initializes the shared rubric set using the same pipeline in Section 2 with an empty memory and the first-step batch. The first-step rollouts are then scored synchronously using the initialized rubric set. In the per-instance setting, each input-specific rubric set is initialized using the annotated rubrics provided by the dataset. For global rubrics, rubric improvement is triggered at every step by default. For per-instance rubrics, updates occur when the corresponding input reappears and new rollouts are available for analysis, since different inputs are sampled at different steps. To increase revisit frequency, we set the batch size to 256 and the number of samples per query to 4. We enable meta summarization for global rubrics with chunk size 32. We set the maximum rubric staleness to 1 version, meaning that each step scores rollouts using rubrics updated from at most one version prior. If the background pipeline has not yet completed, the training loop waits for its result before proceeding. We additionally measure performance under different rubric update intervals in Section E and profile the latency in Section 4.4.

4 Results & Analysis

This section evaluates AMARIS from four angles. Section 4.1 reports downstream results across science, medicine, instruction following, and creative writing. Section 4.2 ablates memory configuration and retrieval budgets. Section 4.3 tests whether online rubric adaptation is needed beyond static initial or final rubric sets. Section 4.4 profiles the pipeline latency and compares synchronous and asynchronous execution. Additional per-instance results, LLM-component ablations, update-frequency controls, scalar-judge baselines, alternative-base-model results, rubric-quality evaluation, rubric-evolution analysis, and case studies appear in the appendix.

4.1 Main Results

We compare AMARIS against general and domain-specific baselines across four evaluation settings. As shown in Table 1, AMARIS consistently achieves the best performance on all benchmarks under both rubric strategies, with the per-instance rubric setting yielding the strongest results on six of the seven benchmarks. Relative to the strongest

Method	Science	Medicine	Instruction Following			Creative Writing	
	GPQA-D	HealthBench	IFEval	InfoBench	IFBench	WritingBench	CW-v3
<i>Cross-domain baselines</i>							
Naive (no RL)	35.0	22.7	77.3	78.1	28.2	45.2	37.4
RubricHub (RuRL)	38.8	33.0	79.8	83.5	33.5	56.9	39.0
RuscaRL	38.5	32.9	79.0	83.2	33.0	56.1	38.6
<i>Science & Medicine baselines</i>							
RaR	37.6	31.2	–	–	–	–	–
<i>Instruction-following baselines</i>							
OpenRubrics (DPO via Rubric-RM)	–	–	79.5	83.0	33.7	–	–
Rubric-ARM	–	–	80.4	83.7	35.0	–	–
RLCF	–	–	78.6	84.1	28.2	–	–
AMARIS (Our proposed system)							
AMARIS (global rubric)	41.1 ± 0.95	35.6 ± 1.18	81.5 ± 0.38	86.6 ± 0.26	36.4 ± 0.58	60.4 ± 0.97	41.7 ± 1.08
AMARIS (per-instance rubric)	41.6 ± 0.39	36.0 ± 1.06	81.9 ± 0.44	87.0 ± 0.23	37.2 ± 0.66	59.8 ± 0.80	42.3 ± 1.48

Table 1: Main results across four evaluation domains. Best results in each column are in **bold**. “–” indicates that the result is not applicable to that domain. Results reported with standard deviations are computed over five repeated evaluations of the same trained checkpoint.

non-AMARIS baseline, the best AMARIS result improves by +2.8 on GPQA-Diamond, +3.0 on HealthBench, +1.5 on IFEval, +2.9 on InfoBench, +2.2 on IFBench, +3.5 on WritingBench, and +3.3 on CW-v3.

4.2 Ablation Studies On Memory Usage

We conduct two sets of experiments to validate the core design choices of AMARIS about using evaluation memory: (i) we isolate the contribution of each memory type by comparing no memory, static-only, dynamic-only, and the combined configuration; and (ii) we change the static memory retrieval number N and dynamic query number K . Results are in Table 2 for global rubrics and Table 7 for per-instance rubrics.

Memory configuration. Both memory types show gains over the no-memory baseline, and their combination yields the best results across all benchmarks (+1.9 on GPQA-D and +1.8 on HealthBench). Notably, the two types have complementary strengths. For example, in scientific domain, static memory slightly outperforms dynamic memory (40.1 vs. 39.8), suggesting that recent historical evaluation steps is valuable for scientific reasoning, possibly because rubric drift and recurring errors tend to appear within a short period of steps. For HealthBench, the pattern (35.3 vs. 35.1) indicates that targeted semantic retrieval is more beneficial due to suboptimal behaviors recur across distant steps, unlike the science domain. In all cases, the combined configuration captures both temporal and

semantic signals and achieves the strongest overall performance, which we adopt as the default, and the data of per-instance rubrics shows similar trend as shown in Table 7.

Retrieval configuration. For both retrieval mechanisms, we observe performance gain up to a moderate budget followed by diminishing returns. Increasing the static retrieval amount from $N=2$ to $N=4$ provides the main improvement (e.g., +0.7 on both GPQA-D and HealthBench), suggesting that four preceding summaries are sufficient to capture the recent training trajectory and prevent oscillatory rubric changes. Similarly, increasing the dynamic retrieval amount by increasing the query from $K=5$ to $K=10$ produces consistent improvements across all benchmarks, but $K=15$ provides no consistent further benefit: most metrics remain flat or decline, while WritingBench improves only slightly (+0.2) and HealthBench drops from 35.3 to 34.9. We attribute this to additional queries retrieving marginal or noisy documents. We therefore select $N=4$ and $K=10$ as defaults. For per-instance rubric settings, the trend agrees with the global rubrics setting as shown in Table 7.

4.3 Static Rubric Spectrum

We further test whether AMARIS gains come from online rubric adaptation or simply from obtaining a better final rubric set. Table 3 reports the global-rubric comparison, and Appendix F.1 reports the corresponding per-instance comparison. The overall trend is initial-static $\leq$ no-memory adaptive

Memory	Setting	Science	Medicine	Instruction Following			Creative Writing	
		GPQA-D	HealthBench	IFEval	InfoBench	IFBench	WritingBench	CW-v3
No memory	–	39.2	33.8	80.6	85.3	35.2	57.3	41.0
Static only	$N = 2$	39.4	34.4	80.8	85.6	35.5	57.9	41.1
	$N = 4$	40.1	35.1	81.1	86.0	35.9	58.6	41.3
	$N = 6$	40.0	34.9	81.0	85.9	35.8	58.5	41.3
	$N = 8$	40.1	35.2	81.1	86.1	36.0	58.5	41.4
Dynamic only	$K = 5$	39.6	34.4	80.9	85.9	35.4	58.0	41.2
	$K = 10$	39.8	35.3	81.0	86.2	36.0	58.2	41.4
	$K = 15$	39.8	34.9	80.9	86.0	35.8	58.4	41.3
Static + Dynamic	$N = 4, K = 10$	41.1	35.6	81.5	86.6	36.4	60.4	41.7

Table 2: Ablation on memory configuration and retrieval hyperparameters under the global rubrics setting. Best results within the static-only and dynamic-only blocks are in **bold**.

Rubric regime	Science	Medicine	Instruction Following			Creative Writing	
	GPQA-D	HealthBench	IFEval	InfoBench	IFBench	WritingBench	CW-v3
Initial-static	38.9	33.7	80.6	85.1	35.2	57.1	40.8
No-memory adaptive	39.1	33.9	80.7	85.2	35.3	57.4	40.9
Final-static	40.9	35.5	81.3	86.3	36.1	59.9	41.7
AMARIS dynamic	41.2	35.5	81.6	86.7	36.3	60.3	41.8

Table 3: Static rubric spectrum under global rubrics. All conditions use the same base checkpoint, training data, RL algorithm, and training budget; only the rubric update regime changes.

$< \text{final-static} \lesssim \text{AMARIS dynamic}$. Frozen cold-start rubrics are consistently weakest because they have not been revised with training feedback. No-memory adaptation gives only small gains over this cold-start condition, indicating that local edits alone are insufficient when the updater cannot retrieve prior diagnostics or previous update outcomes. Final-static rubrics recover most of the improvement, showing that AMARIS learns useful criteria during training, but they remain below or tied with online AMARIS. This residual gap is consistent with the stage-specific behavior observed in Appendix G: early training benefits from defensive criteria, mid-training shifts toward curriculum advancement, and late training enters maintenance. A single frozen final set cannot provide this training-stage curriculum.

4.4 Latency Analysis

A practical concern for any auxiliary reward pipeline is the wall-clock latency added to RL training when the pipeline sits on the critical path. We therefore profile each AMARIS component and compare asynchronous execution in Section 2.4 with traditional synchronous execution, where scoring always waits for the rubric update within the same step so that the rubric stays on-policy.

Component	Avg. time (s)	% of pipeline	Tokens / step
Individual analysis	426.4	63.1%	2.32 M
Batch summarization	192.3	28.5%	246 k
Query generation	2.7	0.4%	14.1 k
Memory retrieval	23.7	3.5%	0
Rubric update	30.6	4.5%	110.9 k
Total pipeline	675.7	100.0%	2.691 M

Table 4: Per-component time and token consumption of the AMARIS averaged over 400 training steps.

Component-level profiling. Table 4 shows the average time and token consumption of each AMARIS component measured over one complete training using the default setting of AMARIS under global rubrics. The individual rollout analysis dominates both time (63.1%) and tokens (2.32 M per step), as it processes every rollout. The batch summarization is the second most expensive stage (28.5%), while the other stages together account for less than 9% of the total pipeline time.

Asynchronous vs. Synchronous execution. Table 5 shows the quality-latency trade-off. Synchronous execution yields mostly a 0.2-point advantage on any single benchmark (e.g., 41.3 vs. 41.1 on GPQA-D). This marginal gain comes at the cost of nearly doubling the total training time. Asynchronous execution therefore provides a substantially more favorable wall-clock trade-off and

Pipeline mode	Science	Medicine	Instruction Following			Creative Writing		Time (h)	Time increase
	GPQA-D	HealthBench	IFEval	InfoBench	IFBench	WritingBench	CW-v3		
Sync	41.3	35.7	81.6	86.4	36.6	60.0	41.8	~146	~92%
Async	41.1	35.6	81.5	86.6	36.4	60.4	41.7	~80	~5%

Table 5: Synchronous vs. asynchronous pipeline execution. Async mode substantially reduces wall-clock latency relative to synchronous execution while incurring only marginal quality differences (≤ 0.2 across most benchmarks).

is adopted as the default pipeline mode. We also compare the total time consumption using both asynchronous and synchronous AMARIS with the normal rubric-based RL training with static rubrics only in terms of the Time increase column in Table 5. The results show that asynchronous execution sharply reduces blocking latency by moving rubric improvement off the training critical path. We also profile the latency for per-instance rubric settings and find that the additional wall-clock delay is small. This is because the per-instance setting naturally limits update opportunity but provides abundant time for analysis and rubric update, especially for summarization and updating that can be further parallelized by using external APIs.

5 Related Work

5.1 Rubric-based RL for LLM

LLM alignment has evolved from scalar reward models trained on pairwise preferences (Ouyang et al., 2022; Bai et al., 2022) toward reward signals that expose more of the underlying rubrics, making the reward specification more interpretable and editable (Kim et al., 2024; Gunjal et al., 2025; Viswanathan et al., 2025). This paradigm extends RL beyond strictly verifiable domains. However, a rubric set is still a reward specification: if it is static, incomplete, or overly coarse, optimization can exploit gaps in the evaluator in the same spirit as reward model overoptimization (Gao et al., 2023). This motivates methods that adapt rubrics as policy behavior changes during training.

5.2 Adaptive Rubrics

Adaptive rubric methods address the static-specification problem by generating or revising criteria during evaluation and training. CARMO (Gupta et al., 2025) and OnlineRubrics (Rezaei et al., 2025) elicit criteria from context or pairwise comparisons, while Rubric-ARM (Xu et al., 2026) and DR Tulu (Shao et al., 2025) couple policy optimization with bounded rubric refinement or evolving rubrics. Related reward-modeling

work reduces reward hacking or guides learning through causal rubric structure (Srivastava et al., 2025), information-theoretic regularization (Miao et al., 2024), and staged AI feedback (Li et al., 2025). These approaches make the reward specification more flexible, but most updates are grounded in local evidence such as the current input, pairwise comparisons, or a predefined curriculum. They typically do not preserve rollout diagnostics, score trends, and previous rubric-edit outcomes as reusable state for later updates.

5.3 Memory for Longitudinal Usage

External memory systems for LLM agents show that persistent state can support long-horizon adaptation. A-Mem (Xu et al., 2025), Memory-R1 (Yan et al., 2026), and Meta-Policy Reflexion (Wu et al., 2025a) introduce read-write-consolidation mechanisms for retaining and reusing experience across tasks, reflecting a broader shift toward memory-augmented agents (Hu et al., 2026). AMARIS extends memory-augmented systems to evaluation-state maintenance, where the stored objects are rollout diagnostics, score trends, and rubric-edit outcomes rather than task-solving experiences. Rubric updates can then use recent trajectories and related past evidence instead of isolated local edits.

6 Conclusion

We present AMARIS, a memory-augmented rubric improvement system for rubric-based reinforcement learning. AMARIS stores rollout analyses, step-level summaries, and rubric update records in a persistent evaluation memory, and uses both static and dynamic retrieval to ground rubric changes in training history. Across science, medicine, instruction following, and creative writing domains, AMARIS improves over strong rubric-based baselines under both global and per-instance rubric settings. Ablation studies show that both retrieval modes contribute to the gains, while asynchronous execution reduces the blocking latency of rubric improvement during training.

Limitations

AMARIS requires additional inference resources to analyze rollouts, summarize batches, retrieve historical evidence, and update rubrics. Asynchronous execution reduces the blocking latency observed by the RL training loop, but it does not reduce the total compute required by these auxiliary stages. Future work could further optimize the update interval and scheduling policy. The update-frequency study in Appendix E shows that updating every 2 steps is slightly better than updating every step on several benchmarks, suggesting room for cost-performance trade-offs.

AMARIS also inherits limitations from rubric-based evaluation. Rubric quality is empirically audited through ablations, evolution analysis, and case studies, but it is not formally guaranteed. The LLM judges used for scoring and diagnostic analysis may encode biases from their training data or from the evaluation instructions. Our RL training experiments primarily use Qwen2.5-7B-Instruct as the policy model and include an additional Qwen3-4B diagnostic, but broader validation across additional base-model families and scales remains future work. Final rubrics may transfer imperfectly across model families, as different policies can expose different failure modes and capability ceilings. Finally, memory retrieval can surface noisy or weakly relevant historical evidence, which may distort rubric updates if the updater fails to filter it.

Ethical Considerations

The authors of this paper have read and adhered to the ACL Code of Ethics. This work focuses on improving the effectiveness of rubric-based reinforcement learning for large language models by grounding rubric adaptation in persistent evaluation memory, and we have considered the ethical implications of our methodology. AMARIS itself is a meta-level training framework that refines evaluation rubrics and does not introduce new model capabilities that pose direct safety risks beyond those of the underlying policy model. Nevertheless, any system that shapes the reward signal of an RL-trained language model could, in principle, amplify undesirable behaviors if the rubrics or LLM judges encode biases present in their training data or evaluation criteria. To mitigate this, AMARIS explicitly monitors for reward hacking and suboptimal behaviors as a core component of its individual rollout analysis, and the persistent evaluation mem-

ory provides an audit record that supports transparency. All training data used in our experiments are drawn from publicly available, previously published datasets. No human subjects were involved in our research, and no personally identifiable information was collected or used.

References

- Rahul K. Arora, Jason Wei, Rebecca Soskin Hicks, Preston Bowman, Joaquin Quiñero-Candela, Foivos Tsimpourlas, Michael Sharman, Meghan Shah, Andrea Vallone, Alex Beutel, Johannes Heidecke, and Karan Singhal. 2025. [Healthbench: Evaluating large language models towards improved human health](#).
- Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Tristan Hume, Scott Johnston, Shauna Kravec, Liane Lovitt, Neel Nanda, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, Ben Mann, and Jared Kaplan. 2022. [Training a helpful and harmless assistant with reinforcement learning from human feedback](#).
- chroma-core. 2026. Chroma: Open-source data infrastructure for ai. <https://github.com/chroma-core/chroma>. GitHub repository, accessed 2026-03-31.
- Kilian Freitag, Kristian Ceder, Rita Laezza, Knut Åkesson, and Morteza Haghir Chehreghani. 2025. [Curriculum reinforcement learning for complex reward functions](#).
- Leo Gao, John Schulman, and Jacob Hilton. 2023. Scaling laws for reward model overoptimization. In *Proceedings of the 40th International Conference on Machine Learning, ICML’23*. JMLR.org.
- Anisha Gunjal, Anthony Wang, Elaine Lau, Vaskar Nath, Yunzhong He, Bing Liu, and Sean Hendryx. 2025. [Rubrics as rewards: Reinforcement learning beyond verifiable domains](#).
- Taneesh Gupta, Shivam Shandilya, Xuchao Zhang, Rahul Madhavan, Supriyo Ghosh, Chetan Bansal, Huaxiu Yao, and Saravan Rajmohan. 2025. [CARMO: Dynamic criteria generation for context aware reward modelling](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 2202–2261, Vienna, Austria. Association for Computational Linguistics.
- Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan,

- Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiaxin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xuanjing Huang, Yu-Gang Jiang, and Shuicheng Yan. 2026. [Memory in the age of ai agents](#).
- Seungone Kim, Jamin Shin, Yejin Cho, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdoo Yun, Seongjin Shin, Sungdong Kim, James Thorne, and Minjoon Seo. 2024. [Prometheus: Inducing fine-grained evaluation capability in language models](#).
- Mengdi Li, Jiaye Lin, Xufeng Zhao, Wenhao Lu, Peilin Zhao, Stefan Wermter, and Di Wang. 2025. [Curriculum-rlaif: Curriculum alignment with reinforcement learning from ai feedback](#).
- Sunzhu Li, Jiale Zhao, Miteto Wei, Huimin Ren, Yang Zhou, Jingwen Yang, Shunyu Liu, Kaike Zhang, and Wei Chen. 2026. [Rubrichub: A comprehensive and highly discriminative rubric dataset via automated coarse-to-fine generation](#).
- Tianci Liu, Ran Xu, Tony Yu, Ilgee Hong, Carl Yang, Tuo Zhao, and Haoyu Wang. 2026. [Openrubrics: Towards scalable synthetic rubric generation for reward modeling and llm alignment](#).
- Yuchun Miao, Sen Zhang, Liang Ding, Rong Bao, Lefei Zhang, and Dacheng Tao. 2024. Inform: mitigating reward hacking in rlhf via information-theoretic reward modeling. In *Proceedings of the 38th International Conference on Neural Information Processing Systems, NIPS '24*, Red Hook, NY, USA. Curran Associates Inc.
- OpenAI, :, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, Haiming Bao, Boaz Barak, Ally Bennett, Tyler Bertao, Nivedita Brett, Eugene Brevdo, Greg Brockman, Sebastien Bubeck, Che Chang, Kai Chen, Mark Chen, Enoch Cheung, Aidan Clark, Dan Cook, Marat Dukhan, Casey Dvorak, Kevin Fives, Vlad Fomenko, Timur Garipov, Kristian Georgiev, Mia Glaese, Tarun Gogineni, Adam Goucher, Lukas Gross, Katia Gil Guzman, John Hallman, Jackie Hehir, Johannes Heidecke, Alec Helyar, Haitang Hu, Romain Huet, Jacob Huh, Saachi Jain, Zach Johnson, Chris Koch, Irina Kofman, Dominik Kundel, Jason Kwon, Volodymyr Kyrylov, Elaine Ya Le, Guillaume Leclerc, James Park Lennon, Scott Lessans, Mario Lezcano-Casado, Yuanzhi Li, Zhuohan Li, Ji Lin, Jordan Liss, Lily, Liu, Jiancheng Liu, Kevin Lu, Chris Lu, Zoran Martinovic, Lindsay McCallum, Josh McGrath, Scott McKinney, Aidan McLaughlin, Song Mei, Steve Mostovoy, Tong Mu, Gideon Myles, Alexander Neitz, Alex Nichol, Jakub Pachocki, Alex Paino, Dana Palmie, Ashley Pantuliano, Giambattista Parascandolo, Jongsoo Park, Leher Pathak, Carolina Paz, Ludovic Peran, Dmitry Pimenov, Michelle Pokrass, Elizabeth Proehl, Huida Qiu, Gaby Raila, Filippo Raso, Hongyu Ren, Kimmy Richardson, David Robinson, Bob Rotsted, Hadi Salman, Suvansh Sanjeev, Max Schwarzer, D. Sculley, Harshit Sikchi, Kendal Simon, Karan Singhal, Yang Song, Dane Stuckey, Zhiqing Sun, Philippe Tillet, Sam Toizer, Foivos Tsimpourlas, Nikhil Vyas, Eric Wallace, Xin Wang, Miles Wang, Olivia Watkins, Kevin Weil, Amy Wendling, Kevin Whinnery, Cedric Whitney, Hannah Wong, Lin Yang, Yu Yang, Michihiro Yasunaga, Kristen Ying, Wojciech Zaremba, Wenting Zhan, Cyril Zhang, Brian Zhang, Eddie Zhang, and Shengjia Zhao. 2025. [gpt-oss-120b & gpt-oss-20b model card](#).
- OpenAI, :, Aaron Hurst, Adam Lerer, Adam P. Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, Aleksander Madry, Alex Baker-Whitcomb, Alex Beutel, Alex Borzunov, Alex Carney, Alex Chow, Alex Kirillov, Alex Nichol, Alex Paino, Alex Renzin, Alex Tachard Passos, Alexander Kirillov, Alexi Christakis, Alexis Conneau, Ali Kamali, Allan Jabri, Allison Moyer, Allison Tam, Amadou Crookes, Amin Tootoochian, Amin Tootoonchian, Ananya Kumar, Andrea Vallone, Andrej Karpathy, Andrew Braunstein, Andrew Cann, Andrew Codispoti, Andrew Galu, Andrew Kondrich, Andrew Tulloch, Andrey Mishchenko, Angela Baek, Angela Jiang, Antoine Pelisse, Antonia Woodford, Anuj Gosalia, Arka Dhar, Ashley Pantuliano, Avi Nayak, Avital Oliver, Barret Zoph, Behrooz Ghorbani, Ben Leimberger, Ben Rossen, Ben Sokolowsky, Ben Wang, Benjamin Zweig, Beth Hoover, Blake Samic, Bob McGrew, Bobby Spero, Bogo Gierler, Bowen Cheng, Brad Lightcap, Brandon Walkin, Brendan Quinn, Brian Guarraci, Brian Hsu, Bright Kellogg, Brydon Eastman, Camillo Lugaresi, Carroll Wainwright, Cary Bassin, Cary Hudson, Casey Chu, Chad Nelson, Chak Li, Chan Jun Shern, Channing Conger, Charlotte Barette, Chelsea Voss, Chen Ding, Cheng Lu, Chong Zhang, Chris Beaumont, Chris Hallacy, Chris Koch, Christian Gibson, Christina Kim, Christine Choi, Christine McLeavey, Christopher Hesse, Claudia Fischer, Clemens Winter, Coley Czarnecki, Colin Jarvis, Colin Wei, Constantin Koumouzelis, Dane Sherburn, Daniel Kappler, Daniel Levin, Daniel Levy, David Carr, David Farhi, David Mely, David Robinson, David Sasaki, Denny Jin, Dev Valladares, Dimitris Tsipras, Doug Li, Duc Phong Nguyen, Duncan Findlay, Edede Oiwoh, Edmund Wong, Ehsan Asdar, Elizabeth Proehl, Elizabeth Yang, Eric Antonow, Eric Kramer, Eric Peterson, Eric Sigler, Eric Wallace, Eugene Brevdo, Evan Mays, Farzad Khorasani, Felipe Petroski Such, Filippo Raso, Francis Zhang, Fred von Lohmann, Freddie Sulit, Gabriel Goh, Gene Oden, Geoff Salmon, Giulio Starace, Greg Brockman, Hadi Salman, Haiming Bao, Haitang Hu, Hannah Wong, Haoyu Wang, Heather Schmidt, Heather Whitney, Heewoo Jun, Hendrik Kirchner, Henrique Ponde de Oliveira Pinto, Hongyu Ren,

- Huiwen Chang, Hyung Won Chung, Ian Kivlichan, Ian O’Connell, Ian O’Connell, Ian Osband, Ian Silber, Ian Sohl, Ibrahim Okuyucu, Ikai Lan, Ilya Kostrikov, Ilya Sutskever, Ingmar Kanitscheider, Ishaan Gulrajani, Jacob Coxon, Jacob Menick, Jakub Pachocki, James Aung, James Betker, James Crooks, James Lennon, Jamie Kiros, Jan Leike, Jane Park, Jason Kwon, Jason Phang, Jason Teplitz, Jason Wei, Jason Wolfe, Jay Chen, Jeff Harris, Jenia Vavva, Jessica Gan Lee, Jessica Shieh, Ji Lin, Jiahui Yu, Jiayi Weng, Jie Tang, Jieqi Yu, Joanne Jang, Joaquin Quinero Candela, Joe Beutler, Joe Landers, Joel Parish, Johannes Heidecke, John Schulman, Jonathan Lachman, Jonathan McKay, Jonathan Uesato, Jonathan Ward, Jong Wook Kim, Joost Huizinga, Jordan Sitkin, Jos Kraaijeveld, Josh Gross, Josh Kaplan, Josh Snyder, Joshua Achiam, Joy Jiao, Joyce Lee, Juntang Zhuang, Justyn Harriman, Kai Fricke, Kai Hayashi, Karan Singhal, Katy Shi, Kevin Karthik, Kayla Wood, Kendra Rimbach, Kenny Hsu, Kenny Nguyen, Keren Gu-Lemberg, Kevin Button, Kevin Liu, Kiel Howe, Krithika Muthukumar, Kyle Luther, Lama Ahmad, Larry Kai, Lauren Itow, Lauren Workman, Leher Pathak, Leo Chen, Li Jing, Lia Guy, Liam Fedus, Liang Zhou, Lien Mamitsuka, Lilian Weng, Lindsay McCallum, Lindsey Held, Long Ouyang, Louis Feuvrier, Lu Zhang, Lukas Kondraciuk, Lukasz Kaiser, Luke Hewitt, Luke Metz, Lyric Doshi, Mada Aflak, Maddie Simens, Madelaine Boyd, Madeleine Thompson, Marat Dukhan, Mark Chen, Mark Gray, Mark Hudnall, Marvin Zhang, Marwan Aljubeih, Mateusz Litwin, Matthew Zeng, Max Johnson, Maya Shetty, Mayank Gupta, Meghan Shah, Mehmet Yatbaz, Meng Jia Yang, Mengchao Zhong, Mia Glaese, Mianna Chen, Michael Janner, Michael Lampe, Michael Petrov, Michael Wu, Michele Wang, Michelle Fradin, Michelle Pokrass, Miguel Castro, Miguel Oom Temudo de Castro, Mikhail Pavlov, Miles Brundage, Miles Wang, Minal Khan, Mira Murati, Mo Bavarian, Molly Lin, Murat Yesildal, Nacho Soto, Natalia Gimelshein, Natalie Cone, Natalie Staudacher, Natalie Summers, Natan LaFontaine, Neil Chowdhury, Nick Ryder, Nick Stathas, Nick Turley, Nik Tezak, Niko Felix, Nithanth Kudige, Nitish Keskar, Noah Deutsch, Noel Bundick, Nora Puckett, Ofir Nachum, Ola Okelola, Oleg Boiko, Oleg Murk, Oliver Jaffe, Olivia Watkins, Olivier Godement, Owen Campbell-Moore, Patrick Chao, Paul McMillan, Pavel Belov, Peng Su, Peter Bak, Peter Bakkum, Peter Deng, Peter Dolan, Peter Hoeschele, Peter Welinder, Phil Tillet, Philip Pronin, Philippe Tillet, Prafulla Dhariwal, Qiming Yuan, Rachel Dias, Rachel Lim, Rahul Arora, Rajan Troll, Randall Lin, Rapha Gontijo Lopes, Raul Puri, Reah Miyara, Reimar Leike, Renaud Gaubert, Reza Zamani, Ricky Wang, Rob Donnelly, Rob Honsby, Rocky Smith, Rohan Sahai, Rohit Ramchandani, Romain Huet, Rory Carmichael, Rowan Zellers, Roy Chen, Ruby Chen, Ruslan Nigmatullin, Ryan Cheu, Saachi Jain, Sam Altman, Sam Schoenholz, Sam Toizer, Samuel Miserendino, Sandhini Agarwal, Sara Culver, Scott Ethersmith, Scott Gray, Sean Grove, Sean Metzger, Shamez Hermani, Shantanu Jain, Shengjia Zhao, Sherwin Wu, Shino Jomoto, Shiron Wu, Shuaiqi, Xia, Sonia Phene, Spencer Papay, Srinivas Narayanan, Steve Coffey, Steve Lee, Stewart Hall, Suchir Balaji, Tal Broda, Tal Stramer, Tao Xu, Tarun Gogineni, Taya Christianson, Ted Sanders, Tejal Patwardhan, Thomas Cunningham, Thomas Degry, Thomas Dimson, Thomas Raoux, Thomas Shadwell, Tianhao Zheng, Todd Underwood, Todor Markov, Toki Sherbakov, Tom Rubin, Tom Stasi, Tomer Kaftan, Tristan Heywood, Troy Peterson, Tyce Walters, Tyna Eloundou, Valerie Qi, Veit Moeller, Vinnie Monaco, Vishal Kuo, Vlad Fomenko, Wayne Chang, Weiye Zheng, Wenda Zhou, Wesam Manassra, Will Sheu, Wojciech Zaremba, Yash Patil, Yilei Qian, Yongjik Kim, Youlong Cheng, Yu Zhang, Yuchen He, Yuchen Zhang, Yujia Jin, Yunxing Dai, and Yury Malkov. 2024. [Gpt-4o system card](#).
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. In *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS ’22*, Red Hook, NY, USA. Curran Associates Inc.
- Samuel J Paech. 2025. Eq-bench creative writing benchmark v3. <https://github.com/EQ-bench/creative-writing-bench>.
- Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hannaneh Hajishirzi. 2025. [Generalizing verifiable instruction following](#).
- Yiwei Qin, Kaiqiang Song, Yebowen Hu, Wenlin Yao, Sangwoo Cho, Xiaoyang Wang, Xuansheng Wu, Fei Liu, Pengfei Liu, and Dong Yu. 2024. [InFoBench: Evaluating instruction following ability in large language models](#). In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 13025–13048, Bangkok, Thailand. Association for Computational Linguistics.
- Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2025. [Qwen2.5 technical report](#).
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2024. [GPQA](#):

[A graduate-level google-proof q&a benchmark](#). In *First Conference on Language Modeling*.

MohammadHossein Rezaei, Robert Vacareanu, Zihao Wang, Clinton Wang, Bing Liu, Yunzhong He, and Afra Feyza Akyürek. 2025. [Online rubrics elicitation from pairwise comparisons](#).

Rulin Shao, Akari Asai, Shannon Zejiang Shen, Hamish Ivison, Varsha Kishore, Jingming Zhuo, Xinran Zhao, Molly Park, Samuel G. Finlayson, David Sontag, Tyler Murray, Sewon Min, Pradeep Dasigi, Luca Soldaini, Faeze Brahman, Wen tau Yih, Tongshuang Wu, Luke Zettlemoyer, Yoon Kim, Hannaneh Hajishirzi, and Pang Wei Koh. 2025. [Dr tulü: Reinforcement learning with evolving rubrics for deep research](#).

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. [Deepseekmath: Pushing the limits of mathematical reasoning in open language models](#).

Leheng Sheng, Wenchang Ma, Ruixin Hong, Xiang Wang, An Zhang, and Tat-Seng Chua. 2026. [Reinforcing chain-of-thought reasoning with self-evolving rubrics](#).

Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, Akshay Nathan, Alan Luo, Alec Helyar, Aleksander Madry, Aleksandr Efremov, Aleksandra Spyra, Alex Baker-Whitcomb, Alex Beutel, Alex Karpenko, Alex Makelov, Alex Neitz, Alex Wei, Alexandra Barr, Alexandre Kirchmeyer, Alexey Ivanov, Alexi Christakis, Alistair Gillespie, Allison Tam, Ally Bennett, Alvin Wan, Alyssa Huang, Amy McDonald Sandjideh, Amy Yang, Ananya Kumar, Andre Saraiva, Andrea Vallone, Andrei Gheorghe, Andres Garcia Garcia, Andrew Braunstein, Andrew Liu, Andrew Schmidt, Andrew Mereskin, Andrew Mishchenko, Andy Applebaum, Andy Rogerson, Ann Rajan, Annie Wei, Anoop Kotha, Anubha Srivastava, Anushree Agrawal, Arun Vijayvergiya, Ashley Tyra, Ashvin Nair, Avi Nayak, Ben Eggers, Bessie Ji, Beth Hoover, Bill Chen, Blair Chen, Boaz Barak, Borys Minaiev, Botao Hao, Bowen Baker, Brad Lightcap, Brandon McKinzie, Brandon Wang, Brendan Quinn, Brian Fioca, Brian Hsu, Brian Yang, Brian Yu, Brian Zhang, Brittany Brenner, Callie Riggins Zetino, Cameron Raymond, Camillo Lugaresi, Carolina Paz, Cary Hudson, Cedric Whitney, Chak Li, Charles Chen, Charlotte Cole, Chelsea Voss, Chen Ding, Chen Shen, Chengdu Huang, Chris Colby, Chris Hallacy, Chris Koch, Chris Lu, Christina Kaplan, Christina Kim, CJ Minott-Henriques, Cliff Frey, Cody Yu, Coley Czarnecki, Colin Reid, Colin Wei, Cory Decareaux, Cristina Scheau, Cyril Zhang, Cyrus Forbes, Da Tang, Dakota Goldberg, Dan Roberts, Dana Palmie, Daniel Kappler, Daniel Levine, Daniel Wright, Dave Leo, David Lin, David Robinson, Declan Grabb, Derek Chen, Derek Lim, Derek Salama, Dibya Bhattacharjee, Dimitris Tsipras, Dinghua Li, Dingli Yu, DJ Strouse, Drew Williams, Dylan Hunn,

Ed Bayes, Edwin Arbus, Ekin Akyurek, Elaine Ya Le, Elana Widmann, Eli Yani, Elizabeth Proehl, Enis Sert, Enoch Cheung, Eri Schwartz, Eric Han, Eric Jiang, Eric Mitchell, Eric Sigler, Eric Wallace, Erik Ritter, Erin Kavanaugh, Evan Mays, Evgenii Nikishin, Fangyuan Li, Felipe Petroski Such, Filipe de Avila Belbute Peres, Filippo Raso, Florent Bekerian, Foivos Tsimplouras, Fotis Chantzis, Francis Song, Francis Zhang, Gaby Raila, Garrett McGrath, Gary Briggs, Gary Yang, Giambattista Parascandolo, Gildas Chabot, Grace Kim, Grace Zhao, Gregory Valiant, Guillaume Leclerc, Hadi Salman, Hanson Wang, Hao Sheng, Haoming Jiang, Haoyu Wang, Haozhun Jin, Harshit Sikchi, Heather Schmidt, Henry Aspegren, Honglin Chen, Huida Qiu, Hunter Lightman, Ian Covert, Ian Kivlichan, Ian Silber, Ian Sohl, Ibrahim Hammoud, Ignasi Clavera, Ikai Lan, Ilge Akkaya, Ilya Kostrikov, Irina Kofman, Isak Etinger, Ishaan Singal, Jackie Hehir, Jacob Huh, Jacqueline Pan, Jake Wilczynski, Jakub Pachocki, James Lee, James Quinn, Jamie Kiros, Janvi Kalra, Jasmyn Samaroo, Jason Wang, Jason Wolfe, Jay Chen, Jay Wang, Jean Harb, Jeffrey Han, Jeffrey Wang, Jennifer Zhao, Jeremy Chen, Jerene Yang, Jerry Tworek, Jesse Chand, Jessica Landon, Jessica Liang, Ji Lin, Jiancheng Liu, Jianfeng Wang, Jie Tang, Jihan Yin, Joanne Jang, Joel Morris, Joey Flynn, Johannes Ferstad, Johannes Heidecke, John Fishbein, John Hallman, Jonah Grant, Jonathan Chien, Jonathan Gordon, Jongsoo Park, Jordan Liss, Jos Kraaijeveld, Joseph Guay, Joseph Mo, Josh Lawson, Josh McGrath, Joshua Vendrow, Joy Jiao, Julian Lee, Julie Steele, Julie Wang, Junhua Mao, Kai Chen, Kai Hayashi, Kai Xiao, Kamyar Salahi, Kan Wu, Karan Sekhri, Karan Sharma, Karan Singhal, Karen Li, Kenny Nguyen, Keren Gu-Lemberg, Kevin King, Kevin Liu, Kevin Stone, Kevin Yu, Kristen Ying, Kristian Georgiev, Kristie Lim, Kushal Tirumala, Kyle Miller, Lama Ahmad, Larry Lv, Laura Clare, Laurance Fauconnet, Lauren Itow, Lauren Yang, Laurentia Romaniuk, Leah Anise, Lee Byron, Leher Pathak, Leon Maksin, Leyan Lo, Leyton Ho, Li Jing, Liang Wu, Liang Xiong, Lien Mamitsuka, Lin Yang, Lindsay McCallum, Lindsey Held, Liz Bourgeois, Logan Engstrom, Lorenz Kuhn, Louis Feuvrier, Lu Zhang, Lucas Switzer, Lukas Kondraciuk, Lukasz Kaiser, Manas Joglekar, Mandeep Singh, Mandip Shah, Manuka Stratta, Marcus Williams, Mark Chen, Mark Sun, Marselus Cayton, Martin Li, Marvin Zhang, Marwan Aljubei, Matt Nichols, Matthew Haines, Max Schwarzer, Mayank Gupta, Meghan Shah, Melody Huang, Meng Dong, Mengqing Wang, Mia Glaese, Micah Carroll, Michael Lampe, Michael Malek, Michael Sharman, Michael Zhang, Michele Wang, Michelle Pokrass, Mihai Florian, Mikhail Pavlov, Miles Wang, Ming Chen, Mingxuan Wang, Minnia Feng, Mo Bavarian, Molly Lin, Moose Abdool, Mostafa Rohaninejad, Nacho Soto, Natalie Staudacher, Natan LaFontaine, Nathan Marwell, Nelson Liu, Nick Preston, Nick Turley, Nicklas Ansman, Nicole Blades, Nikil Pancha, Nikita Mikhaylin, Niko Felix, Nikunj Handa, Nishant Rai, Nitish Keskar, Noam Brown, Ofir Nachum, Oleg Boiko, Oleg Murk,

- Olivia Watkins, Oona Gleeson, Pamela Mishkin, Patryk Lesiewicz, Paul Baltescu, Pavel Belov, Peter Zhokhov, Philip Pronin, Phillip Guo, Phoebe Thacker, Qi Liu, Qiming Yuan, Qinghua Liu, Rachel Dias, Rachel Puckett, Rahul Arora, Ravi Teja Mullapudi, Raz Gaon, Reah Miyara, Rennie Song, Rishabh Agarwal, RJ Marsan, Robel Yemiru, Robert Xiong, Rohan Kshirsagar, Rohan Nuttall, Roman Tsiupa, Ronen Eldan, Rose Wang, Roshan James, Roy Ziv, Rui Shu, Ruslan Nigmatullin, Saachi Jain, Saam Talaie, Sam Altman, Sam Arnesen, Sam Toizer, Sam Toyer, Samuel Miserendino, Sandhini Agarwal, Sarah Yoo, Savannah Heon, Scott Ethersmith, Sean Grove, Sean Taylor, Sebastien Bubeck, Sever Banesiu, Shaokyi Amdo, Shengjia Zhao, Sherwin Wu, Shibani Santurkar, Shiyu Zhao, Shraman Ray Chaudhuri, Shreyas Krishnaswamy, Shuaiqi, Xia, Shuyang Cheng, Shyamal Anadkat, Simón Posada Fishman, Simon Tobin, Siyuan Fu, Somay Jain, Song Mei, Sonya Egoian, Spencer Kim, Spug Golden, SQ Mah, Steph Lin, Stephen Imm, Steve Sharpe, Steve Yadlowsky, Sulman Choudhry, Sungwon Eum, Suvansh Sanjeev, Tabarak Khan, Tal Stramer, Tao Wang, Tao Xin, Tarun Gogineni, Taya Christianson, Ted Sanders, Tejal Patwardhan, Thomas Degry, Thomas Shadwell, Tianfu Fu, Tianshi Gao, Timur Garipov, Tina Sriskandarajah, Toki Sherbakov, Tomer Kaftan, Tomo Hiratsuka, Tongzhou Wang, Tony Song, Tony Zhao, Troy Peterson, Val Kharitonov, Victoria Chernova, Vineet Kosaraju, Vishal Kuo, Vitchyr Pong, Vivek Verma, Vlad Petrov, Wanning Jiang, Weixing Zhang, Wenda Zhou, Wenlei Xie, Wenting Zhan, Wes McCabe, Will DePue, Will Ellsworth, Wulfie Bain, Wyatt Thompson, Xiangning Chen, Xiangyu Qi, Xin Xiang, Xinwei Shi, Yann Dubois, Yaodong Yu, Yara Khakbaz, Yifan Wu, Yilei Qian, Yin Tat Lee, Yinbo Chen, Yizhen Zhang, Yizhong Xiong, Yonglong Tian, Young Cha, Yu Bai, Yu Yang, Yuan Yuan, Yuanzhi Li, Yufeng Zhang, Yuguang Yang, Yujia Jin, Yun Jiang, Yunyun Wang, Yushi Wang, Yutian Liu, Zach Stuebenvoll, Zehao Dou, Zheng Wu, and Zhigang Wang. 2025. [Openai gpt-5 system card](#).
- Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. 2022. Defining and characterizing reward hacking. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS ’22, Red Hook, NY, USA. Curran Associates Inc.
- Pragya Srivastava, Harman Singh, Rahul Madhavan, Gandharv Patil, Sravanti Addepalli, Arun Suggala, Rengarajan Aravamudhan, Soumya Sharma, Anirban Laha, Aravindan Raghuvier, Karthikeyan Shanmugam, and Doina Precup. 2025. [Robust reward modeling via causal rubrics](#).
- Vijay Viswanathan, Yanchao Sun, Shuang Ma, Xiang Kong, Meng Cao, Graham Neubig, and Tongshuang Wu. 2025. [Checklists are better than reward models for aligning language models](#).
- Chunlong Wu, Ye Luo, Zhibo Qu, and Min Wang. 2025a. [Meta-policy reflexion: Reusable reflective memory and rule admissibility for resource-efficient llm agent](#).
- Yuning Wu, Jiahao Mei, Ming Yan, Chenliang Li, Shaopeng Lai, Yuran Ren, Zijia Wang, Ji Zhang, Mengyue Wu, Qin Jin, and Fei Huang. 2025b. [Writingbench: A comprehensive benchmark for generative writing](#).
- Ran Xu, Tianci Liu, Zihan Dong, Tony Yu, Ilgee Hong, Carl Yang, Linjun Zhang, Tao Zhao, and Haoyu Wang. 2026. [Alternating reinforcement learning for rubric-based reward modeling in non-verifiable llm post-training](#).
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. [A-mem: Agentic memory for llm agents](#).
- Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z. Pan, Hinrich Schütze, Volker Tresp, and Yunpu Ma. 2026. [Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning](#).
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. [Qwen3 technical report](#).
- Zihuiwen Ye, Fraser David Greenlee, Max Bartolo, Phil Blunsom, Jon Ander Campos, and Matthias Gallé. 2025. [Improving reward models with synthetic critiques](#). In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 4506–4520, Albuquerque, New Mexico. Association for Computational Linguistics.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS ’23, Red Hook, NY, USA. Curran Associates Inc.
- Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. [Instruction-following evaluation for large language models](#).
- Yang Zhou, Sunzhu Li, Shunyu Liu, Wenkai Fang, Kongcheng Zhang, Jiale Zhao, Jingwen Yang, Yihe Zhou, Jianwei Lv, Tongya Zheng, Hengtong Lu, Wei

Chen, Yan Xie, and Mingli Song. 2026. [Breaking the exploration bottleneck: Rubric-scaffolded reinforcement learning for general llm reasoning.](#)

Appendix

A Formal AMARIS Procedure

Let s denote a rubric scope. For global rubrics, $s = \text{global}$. For per-instance rubrics, $s = x$ for a unique input. Each scope maintains an active rubric set G_t^s and an evaluation memory M_t^s .

For rollout $(x_{t,i}, y_{t,i})$ in scope s , scoring and diagnosis follow

$$r_{t,i} = S_\theta(x_{t,i}, y_{t,i}, G_t^s),$$

$$a_{t,i} = A_\psi(x_{t,i}, y_{t,i}, G_t^s, z_{t,i}).$$

Let $A_t^s = \{a_{t,i}\}_{i=1}^{m_t^s}$ be the set of rollout analyses for scope s at step t . The step summary is

$$b_t^s = B_\psi(A_t^s).$$

When chunking is used, with chunks $\{c_{t,k}^s\}_{k=1}^{K_t^s}$, the final summary is

$$b_t^s = C_\psi(\{B_\psi(c_{t,k}^s)\}_{k=1}^{K_t^s}).$$

Static retrieval and dynamic retrieval are

$$I_{t,\text{static}}^s = \{b_\tau^s \in M_t^s \mid t - N \leq \tau < t\},$$

$$Q_t^s = Q_\psi(b_t^s),$$

$$I_{t,\text{dynamic}}^s = \bigcup_{q \in Q_t^s} R(q, M_t^s, D).$$

The updater receives

$$I_t^s = I_{t,\text{static}}^s \cup I_{t,\text{dynamic}}^s$$

and produces

$$G_{t+1}^s, u_t^s = U_\psi(G_t^s, b_t^s, I_t^s).$$

Let $\mathcal{C}_t^s = \{B_\psi(c_{t,k}^s)\}_{k=1}^{K_t^s}$ when chunking is used and $\mathcal{C}_t^s = \emptyset$ otherwise. Memory is updated as

$$M_{t+1}^s = M_t^s \cup A_t^s \cup \mathcal{C}_t^s \cup \{b_t^s, u_t^s\}.$$

B Symbols

Symbol	Description
π_ϕ	Policy model.
x	Model input.
y	Model output (rollout), with $y \sim \pi_\phi(\cdot x)$.
x_j	Input for example j .
y_j	Output for example j .
t	RL training step.
s	Rubric scope, either global or per-instance.
G_t	Active rubric set at step t .
G_t^s	Active rubric set for scope s at step t .
g_j	Natural-language definition of rubric j .
w_j	Scalar weight of rubric j .
$n_{g,t}$	Number of active rubrics at step t .
$G_t = \{(g_j, w_j)\}_{j=1}^{n_{g,t}}$	Formal definition of the active rubric set.
S_θ	LLM-based scoring function.
$r_t(x_j, y_j)$	Scalar reward for example j at step t .
$r_t(x_j, y_j) = S_\theta(x_j, y_j, G_t)$	Reward computed under the active rubric set.
z	Supplemental context for analysis.
A_ψ	LLM for individual rollout analysis.
a	Structured analysis for a single rollout.
$a = A_\psi(x, y, G_t, z)$	Individual analysis function.
a_j	Structured analysis for rollout j .
$n_{a,t}$	Number of analyzed rollouts at step t .
$\{a_j\}_{j=1}^{n_{a,t}}$	Set of rollout analyses at step t .
B_ψ	LLM for batch summarization.
b_t	Step-level batch summary at step t .
b_t^s	Step-level batch summary for scope s at step t .
$b_t = B_\psi(\{a_j\}_{j=1}^{n_{a,t}})$	Step-level summarization function.
c_j	Chunk of analyses used in meta-summarization.
$n_{c,t}$	Number of chunks at step t .
$n_{a,c}$	Number of analyses in chunk c .
$\{c_j\}_{j=1}^{n_{c,t}}$	Partition of analyses into chunks.
C_ψ	LLM for meta-summarization.
$C_\psi(\{B_\psi(c_j)\}_{j=1}^{n_{c,t}})$	Meta-summarization over chunk summaries.
M_t	Persistent evaluation memory available at step t .
M_t^s	Persistent evaluation memory for scope s available at step t .
u_t	Rubric improvement record produced at step t .
I_t	Retrieved historical context for rubric updating.
I_t^s	Retrieved historical context for scope s .
$I_{t,\text{static}}^s$	Static retrieval results for scope s .
$I_{t,\text{dynamic}}^s$	Dynamic retrieval results for scope s .
$I_t^s = I_{t,\text{static}}^s \cup I_{t,\text{dynamic}}^s$	Combined retrieval context for scope s .
N	Static retrieval window size.
$I_{t,\text{static}}^s = \{b_\tau^s \in M_t^s \mid t - N \leq \tau < t\}$	Static retrieval from recent summaries.
Q_ψ	LLM for dynamic query generation.
Q_t^s	Dynamic retrieval queries for scope s at step t .
q	Dynamic retrieval query.
$Q_t^s = Q_\psi(b_t^s)$	Queries generated from the current summary.
D	Maximum documents returned per query.
$R(q, M_t^s, D)$	Retrieval operator over memory.
$I_{t,\text{dynamic}}^s = \bigcup_{q \in Q_t^s} R(q, M_t^s, D)$	Dynamic retrieval from memory.
U_ψ	Rubric updater.
G_{t+1}	Updated rubric set after step t .

Table 6: Description of symbols and notations used.

C Per-Instance Rubric Results

The ablation studies in the main paper (Sections 4.2–4.4) use the global rubric setting to isolate the effect of each design choice. This section reports the corresponding results under per-instance rubrics, where each unique input maintains its own rubric set and evaluation memory. As shown below, the relative ordering of configurations is consistent with the global setting, confirming that the design conclusions drawn in the main paper generalize across rubric strategies.

C.1 Memory Configuration (Per-Instance)

Table 7 repeats the memory ablation under per-instance rubrics. The same pattern as the global setting holds: combining static and dynamic memory gives the strongest overall results, while dynamic-only retrieval with $K=10$ is the best single-memory configuration.

C.2 LLM Choice (Per-Instance)

Table 8 reports the per-instance counterpart of the LLM-component ablation in Table 10. The results show that the per-instance setting is also sensitive to the quality of the scoring/analysis and summarization/update models, while the default GPT-OSS-120B + GPT-5.1 configuration remains competitive across domains.

C.3 Synchronous vs. Asynchronous Pipeline (Per-Instance)

Table 9 compares synchronous and asynchronous execution under per-instance rubrics. As in the global setting, synchronous execution gives only marginal quality gains, while asynchronous execution substantially reduces wall-clock time and is therefore the more practical default.

D Influence Of Underlying LLMs

We split all LLM usage into two tiers: (i) scoring and analysis that processes every rollout and therefore dominates token consumption, and (ii) summarization and update that operates less than (i) but demands stronger reasoning for rubric improvement. We ablate each tier independently while holding the other at its default, as shown in Table 10.

On the scoring/analysis side, the larger model improves several benchmarks but not uniformly. Upgrading from Qwen3-30B-A3B-2507-Thinking

(Yang et al., 2025) to Qwen3-235B-A22B-2507-Thinking (Yang et al., 2025) improves GPQA-D by +0.6 and InfoBench by +2.1, although HealthBench and CW-v3 do not improve. This suggests that stronger individual analyses can help downstream rubric updates, but benchmark-specific evaluator fit also matters. On the summarization/update side, GPT-5.1 consistently outperforms GPT-4o (OpenAI et al., 2024), showing that the rubric update stage, which must make precise text and weight editing decisions, benefits from stronger reasoning ability of the model. Notably, GPT-5-mini (Singh et al., 2025) closes much of this gap to GPT-5.1 at lower cost and surpasses GPT-4o on some benchmarks. This shows that the reasoning capability is crucial to the quality of generated rubrics. We select GPT-OSS-120B paired with GPT-5.1 as the default setting as it offers a balance between the high inference cost from scoring/analysis stage and high demand of model intelligence from summarization/update stage.

E Influence Of Update Scheduling

By default, under global rubrics setting, the rubric update stage is triggered at every training step. We vary the update interval from every 1 to every 8 steps to examine how update frequency affects downstream performance, as shown in Table 11.

Updating every 2 steps yields the best or near-best results on most of the benchmarks, marginally outperforming every-step updates (e.g., +0.2 on GPQA-D, +0.1 on HealthBench). We attribute this to the slightly larger evidence window that aggregating two consecutive batches before committing a rubric change produces more robust diagnostics than a single batch while still reacting quickly to emerging issues. Performance degrades at intervals of 4 and 6 steps, where the difference between the rubrics and the model becomes large enough to miss behavioral shifts. Interestingly, the 8-step interval setting partially recovers the performance. A possible explanation is that the larger amount of historical analyses at longer intervals enables the updater to make reliable rubric changes that are robust to long-term training, which particularly benefits the diverse instruction types in the OpenRubrics training set. However, this may result in slower reaction to suboptimal behaviors, as reflected by the lower GPQA-D and HealthBench scores. Overall, six of the seven benchmarks vary by at most 0.7 points across update frequencies, while Writing-

Memory	Setting	Science	Medicine	Instruction Following			Creative Writing	
		GPQA-D	HealthBench	IFEval	InfoBench	IFBench	WritingBench	CW-v3
No memory	–	39.6	34.1	80.9	85.6	35.7	57.7	41.2
Static only	$N = 2$	39.9	34.8	81.1	85.9	36.2	57.5	41.5
	$N = 4$	39.8	34.5	81.0	85.8	36.6	58.2	40.8
	$N = 6$	39.5	34.3	80.4	86.3	36.5	58.1	41.8
	$N = 8$	39.7	34.6	80.5	86.5	36.6	58.2	41.7
Dynamic only	$K = 5$	40.2	34.8	81.2	86.2	36.3	58.6	41.6
	$K = 10$	41.1	36.0	81.5	86.6	36.9	59.3	42.3
	$K = 15$	40.4	35.4	81.3	86.4	36.5	58.9	41.8
Static + Dynamic	$N = 4, K = 10$	41.6	36.0	81.9	87.0	37.2	59.8	42.3

Table 7: Ablation on memory configuration and retrieval hyperparameters under per-instance rubrics. Rows are ordered as no memory, static-only, dynamic-only, and static+dynamic. Best results within the static-only and dynamic-only blocks are in **bold**.

Scoring / Analysis	Summarization / Update	Science	Medicine	Instruction Following			Creative Writing	
		GPQA-D	HealthBench	IFEval	InfoBench	IFBench	WritingBench	CW-v3
Varying the scoring / analysis (summarization / update fixed to GPT-5.1)								
Qwen3-30B-A3B-2507-Thinking	GPT-5.1	41.9	35.8	81.1	85.3	37.8	58.3	41.5
GPT-OSS-120B	GPT-5.1	41.6	36.0	81.9	87.0	37.2	59.8	42.3
Qwen3-235B-A22B-2507-Thinking	GPT-5.1	41.1	36.3	82.2	87.3	36.3	60.8	42.6
Varying the summarization / update (scoring / analysis fixed to GPT-OSS-120B)								
GPT-OSS-120B	GPT-4o	41.1	35.4	80.3	86.7	36.1	61.1	43.1
GPT-OSS-120B	GPT-5-mini	41.4	35.8	81.7	86.0	37.0	58.7	41.6
GPT-OSS-120B	GPT-5.1	41.6	36.0	81.9	87.0	37.2	59.8	42.3

Table 8: LLM choice for pipeline components under per-instance rubrics. Compare with Table 10 (global rubrics). Best results per group are in **bold**.

Pipeline mode	Science	Medicine	Instruction Following			Creative Writing		Time (h)	Time increase
	GPQA-D	HealthBench	IFEval	InfoBench	IFBench	WritingBench	CW-v3		
Sync	41.8	36.1	82.0	87.1	37.4	59.6	42.4	~127	~67%
Async	41.6	36.0	81.9	87.0	37.2	59.8	42.3	~78	~3%

Table 9: Synchronous vs. asynchronous pipeline execution under per-instance rubrics. Compared with Table 5 (global rubrics).

Scoring / Analyzing	Summarization / Update	Science	Medicine	Instruction Following			Creative Writing	
		GPQA-D	HealthBench	IFEval	InfoBench	IFBench	WritingBench	CW-v3
Varying the scoring and analysis (summarization / update fixed to GPT-5.1)								
Qwen3-30B-A3B-2507-Thinking	GPT-5.1	40.8	35.9	80.7	84.8	36.4	57.6	42.0
GPT-OSS-120B	GPT-5.1	41.1	35.6	81.5	86.6	36.4	60.4	41.7
Qwen3-235B-A22B-2507-Thinking	GPT-5.1	41.4	35.4	81.8	86.9	37.0	60.3	41.7
Varying the summarizing / updating (scoring / analyzing fixed to GPT-OSS-120B)								
GPT-OSS-120B	GPT-4o	40.6	35.0	79.9	85.6	35.4	58.2	41.2
GPT-OSS-120B	GPT-5-mini	40.9	35.4	81.3	86.3	36.2	59.4	41.5
GPT-OSS-120B	GPT-5.1	41.1	35.6	81.5	86.6	36.4	60.4	41.7

Table 10: LLM choice for pipeline components. Best results per group are in **bold**.

Update interval	Science	Medicine	Instruction Following			Creative Writing	
	GPQA-D	HealthBench	IFEval	InfoBench	IFBench	WritingBench	CW-v3
Every step	41.1	35.6	81.5	86.6	36.4	60.4	41.7
Every 2 steps	41.3	35.7	81.6	86.7	36.6	59.5	41.9
Every 4 steps	41.0	35.5	81.4	86.5	36.2	60.0	41.6
Every 6 steps	40.6	35.2	81.1	86.2	35.9	59.1	41.3
Every 8 steps	40.9	35.3	81.7	86.8	36.3	59.7	41.7

Table 11: Effect of rubric update frequency. Best results per column are in **bold**.

Bench varies by up to 1.3 points, indicating that AMARIS is generally robust to the choice of update frequency. We adopt every-step updates as the default for global rubrics setting due to its simplicity and consistently competitive performance across all domains.

F Additional Diagnostic Experiments

This section reports diagnostic controls that isolate whether AMARIS gains come from dynamic rubric adaptation, explicit rubric decomposition, robustness to the policy base model, and improved rubric quality. These experiments are designed to separate four explanations for the main results: whether a fixed rubric set is already sufficient, whether a strong scalar judge can replace rubric decomposition, whether the gains depend on the default policy base model, and whether the updated rubrics are independently judged to be better.

F.1 Static Rubric Spectrum

Table 12 reports the per-instance counterpart to the global static-rubric spectrum in Table 3. The four regimes keep the RL algorithm, base checkpoint, scoring model, training data, and training budget fixed while changing only how rubrics are initialized and updated. The initial-static condition freezes the cold-start rubric set. The no-memory adaptive condition permits local rubric updates without retrieving historical evidence. The final-static condition reuses the final rubric set produced by a full AMARIS run but freezes it from the start of training. AMARIS dynamic keeps the complete memory-augmented update process.

Cold-start and memory effects. The static spectrum shows that a frozen cold-start rubric is not sufficient. Per-instance rubrics follow the same ordering as global rubrics, with AMARIS dynamic improving over initial-static by +2.0 on GPQA-D, +1.9 on HealthBench, +1.7 on InfoBench, and +2.3 on WritingBench. Allowing local updates without memory gives only small improvements over initial-static, indicating that adaptation itself is not enough when the updater cannot retrieve prior rollout diagnostics and previous update outcomes.

Final rubrics versus dynamic adaptation. Final-static rubrics recover most of the improvement, which indicates that AMARIS learns useful evaluation criteria during training. However, the dynamic condition remains best or tied best

across all columns. Under per-instance rubrics, dynamic adaptation improves over final-static by +0.3 on GPQA-D, +0.2 on HealthBench, +0.1 on IFEval, +0.4 on InfoBench, and +0.2 on WritingBench. Together with the global results in Table 3, this suggests that AMARIS is not merely discovering a better final rubric set; the stage-specific co-evolution between the policy and the rubric set also contributes to the final performance.

F.2 Alternative Reward Interfaces

Table 13 evaluates whether a strong scalar judge can replace rubric-decomposed scoring. Both scalar baselines use the same scoring model as AMARIS but remove explicit rubric criteria. Scalar-Goal provides only the high-level training goal, input, and model output to the judge. Scalar-Goal+Ref additionally provides the reference answer when available.

Effect of reference conditioning. Providing reference answers improves the scalar judge on most benchmarks: Scalar-Goal+Ref improves over Scalar-Goal by +1.0 on GPQA-D, +0.9 on HealthBench, +1.2 on InfoBench, +0.8 on IFEval, +1.8 on WritingBench, and +0.6 on CW-v3. This confirms that the scalar judge can use additional task information when it is available.

Gap to rubric-decomposed scoring. Even with references, scalar scoring remains below AMARIS dynamic across all benchmarks. Relative to the global AMARIS dynamic condition in Table 3, Scalar-Goal+Ref is lower by 2.0 points on GPQA-D, 2.0 on HealthBench, 1.6 on IFEval, 1.9 on InfoBench, 2.0 on IFEval, 3.9 on WritingBench, and 1.2 on CW-v3. This suggests that the gains are not explained solely by access to a strong evaluator or a reference answer. Explicit rubric decomposition provides a more structured reward signal for RL, especially on open-ended writing and heterogeneous instruction-following tasks.

F.3 Alternative Base Model

Table 14 evaluates whether the same conclusion holds when the policy base is changed from the default Qwen2.5-7B-Instruct model to Qwen3-4B (Yang et al., 2025). We compare the no-RL anchor, two scalar-judge reward interfaces, and AMARIS under both global and per-instance rubrics.

Robustness to the policy base. The Qwen3-4B results show that AMARIS gains are not

Rubric regime	Science	Medicine	Instruction Following			Creative Writing	
	GPQA-D	HealthBench	IFEval	InfoBench	IFBench	WritingBench	CW-v3
Initial-static	39.5	34.0	80.7	85.4	35.6	57.6	41.0
No-memory adaptive	39.5	34.2	80.8	85.5	35.8	57.6	41.3
Final-static	41.2	35.7	81.7	86.7	37.1	59.7	42.2
AMARIS dynamic	41.5	35.9	81.8	87.1	37.1	59.9	42.2

Table 12: Static rubric spectrum under per-instance rubrics. All conditions use the same base checkpoint, training data, RL algorithm, and training budget; only the rubric update regime changes.

Reward interface	Science	Medicine	Instruction Following			Creative Writing	
	GPQA-D	HealthBench	IFEval	InfoBench	IFBench	WritingBench	CW-v3
Scalar-Goal	38.2	32.6	79.9	83.6	33.5	54.6	40.0
Scalar-Goal+Ref	39.2	33.5	80.0	84.8	34.3	56.4	40.6

Table 13: Scalar judge reward baselines. Both rows use GPT-OSS-120B as the scalar judge and keep the remaining RL setup fixed.

Method	Science	Medicine	Instruction Following			Creative Writing	
	GPQA-D	HealthBench	IFEval	InfoBench	IFBench	WritingBench	CW-v3
Naive (no RL)	45.1	37.0	80.9	81.6	22.7	56.0	40.8
Scalar-Goal	46.2	59.6	79.3	86.7	29.6	68.5	41.2
Scalar-Goal+Ref	47.2	60.7	79.2	88.2	29.9	70.9	42.0
AMARIS (global rubric)	49.7	62.9	80.9	90.2	32.0	75.0	42.4
AMARIS (per-instance rubric)	50.1	63.5	81.2	90.2	33.2	73.8	43.0

Table 14: Alternative policy-base results with Qwen3-4B. Best results in each column are in **bold**.

tied to the default Qwen2.5-7B-Instruct policy base. AMARIS is best or tied best on all seven benchmarks, with average scores of 61.9 under global rubrics and 62.1 under per-instance rubrics, compared with 52.0 for the no-RL anchor and 59.7 for Scalar-Goal+Ref. Relative to the strongest scalar baseline, AMARIS improves by +2.5/+2.9 on GPQA-D, +2.2/+2.8 on HealthBench for global/per-instance rubrics, respectively. The relative behavior of global and per-instance rubrics is also consistent with the main results: per-instance rubrics are strongest on most benchmarks, while global rubrics are better on WritingBench. This suggests that per-instance rubrics help when prompts expose distinct correctness or instruction-following requirements, whereas a shared global rubric can remain competitive for open-ended writing where broad style and quality criteria transfer across prompts. The IFEval column also illustrates a ceiling effect: the no-RL Qwen3-4B anchor is already strong, scalar rewards slightly reduce the score, and AMARIS recovers or improves it while producing larger gains on InfoBench, IFBench, and WritingBench.

F.4 Rubric Quality Preference Evaluation

Table 15 reports an automated rubric-quality evaluation. We use GPT-5.4 (Singh et al., 2025) as the rubric-quality judge. For each evaluation item, the judge is given the task context and three anonymized candidate rubric sets: the initial rubric set, the final rubric set produced by no-memory adaptation, and the final rubric set produced by AMARIS. Candidate order is randomized, and method identities are hidden. The judge assigns 1–10 scores for six rubric-quality dimensions and then selects the best overall rubric set. Figure 2 provides the evaluation prompt. The final column of Table 15 reports the fraction of identity-hidden comparisons in which each rubric set is selected as best.

Rubric quality improvement. The evaluation indicates that AMARIS final rubrics are preferred more often and receive higher scores on every rubric-quality dimension. Compared with the no-memory final rubrics, AMARIS final rubrics improve coverage by +0.9, clarity by +1.1, discriminativeness by +1.5, non-redundancy by +0.6, exploit resistance by +0.8, and task alignment by +0.9.

Rubric-quality evaluation prompt.

Input. You are given a task context and several anonymized candidate rubric sets. The candidate order is randomized, and method identities are not shown.

Instruction. Evaluate each candidate rubric set independently. For each candidate, assign a 1–10 score for:

1. **Coverage:** captures the important success and failure modes for the task.
2. **Clarity:** uses specific, unambiguous, and easy-to-apply criteria.
3. **Discriminativeness:** separates strong, mediocre, and weak responses.
4. **Non-redundancy:** avoids unnecessary overlap among criteria.
5. **Exploit resistance:** avoids rewarding superficial shortcuts or reward-hacking behavior.
6. **Task alignment:** matches the intended task objective.

Decision. After scoring all candidates, select the single best rubric set overall and provide a brief rationale. Do not use candidate order, length, or inferred method identity as evidence.

Figure 2: Prompt used for automated rubric-quality preference evaluation. The full input additionally includes the task context and anonymized candidate rubric sets.

Rubric set	Coverage	Clarity	Discriminativeness	Non-redundancy	Exploit resistance	Task alignment	Preference
Initial	6.5	6.8	6.1	6.2	5.8	6.2	0.08
No-memory final	7.7	7.9	7.2	7.7	7.3	7.8	0.24
AMARIS final	8.6	9.0	8.7	8.3	8.1	8.7	0.68

Table 15: Automated rubric quality evaluation. Dimension scores use a 1–10 scale; preference is the fraction of identity-hidden comparisons in which the rubric set is selected as best.

The preference share also increases from 0.24 for no-memory final rubrics to 0.68 for AMARIS final rubrics.

Interpretation. These results support the mechanism suggested by the downstream ablations: memory does not only stabilize rubric updates, but also helps the updater produce rubrics that are clearer, less redundant, more discriminative, and more resistant to exploitation. The evaluation is independent of the final benchmark scores, so it provides complementary evidence that AMARIS improves the rubric artifact itself rather than only tuning rewards toward the reported evaluations.

G Rubric Evolvment Analysis

To understand how AMARIS improves the rubrics over the RL training, we analyze 1,200 step-level summaries produced across the three completed training processes under global-rubric settings (science, medicine, and instruction following). Following the three update strategies defined in Section 2.3, we assign each step one dominant update mode (**Defensive**, **Curriculum Advancement**, or **Maintenance**) and partition training into five phases with equal number of steps for chronological comparison. We then taxonomize the dominant pattern within each mode for further analysis.

Three-stage progression. Table 16 reports the distribution of update modes across training phases. AMARIS shows a clear three-stage pattern. In the first 80 steps, **Defensive** updates dominate, indicating that the system initially deals with suboptimal behaviors. In mid-training (steps 81–240), **Curriculum Advancement** becomes the single largest mode, showing that once the suboptimal behaviors are controlled, AMARIS shifts toward raising evaluation standards. In the final phase (steps 241–400), **Maintenance** dominates, indicating that the rubric set becomes stable.

Suboptimal behavior taxonomy. To understand the major suboptimal behaviors, we analyze each of the 334 defensive steps’ batch summarizations by choosing the dominant suboptimal behavior category reported in the summarization as a representative and categorize them manually into six types of major suboptimal behaviors. Table 17 reports the temporal evolution of each type. We define the six types as follows: *length gaming* increases response length to exploit length-correlated reward signals; *superficial format compliance* complies to surface-level formatting such as bullet points, numbered lists, or markdown headers without improving substantive content; *safety over-refusal* unnecessarily declines on user instructions due to overly cautious; *claims without evidence* presents assertions or conclusions without supporting reasoning or citations; *sycophancy* excessively agrees with the user rather than providing accurate responses; and *keyword stuffing* inserts task-relevant terminology or phrases into the output without meaningful integration into the reasoning. Three classic suboptimal behaviors: length gaming and verbosity inflation (23.35%), superficial format compliance (21.26%), and safety over-refusal (18.86%), together account for the ma-

jority of defensive steps (63.47%), showing that most of the early rubric updates have been used to resolve reward hacking against visible rubric cues. All six suboptimal behaviors decrease over training in different rates. Length gaming and sycophancy decrease faster, indicating that once rubric patches penalize these behaviors, the policy quickly learns to avoid them. On the contrary, Safety over-refusal and Keyword stuffing are the most persistent suboptimal behavior for the whole RL training. Keyword stuffing also does not decrease monotonically like the other types, suggesting that it can re-emerge in different forms, which highlights the value of AMARIS’s memory-based tracking of recurring behaviors.

Curriculum advancement taxonomy. Similar to suboptimal behaviors for **Defensive** steps, we use the same method to analyze the main factor for **Curriculum Advancement** steps. Table 18 reports the dominant factors for each of the 413 samples flagged with **Curriculum Advancement**. The six factors are: *reasoning depth*, requiring multi-step or more sophisticated chains of reasoning; *conciseness*, favoring concise but information-dense responses over verbose and vague ones; *justification*, grounding claims in explicit evidence or citations; *trade-off handling*, appropriately balancing different objectives within a single query (e.g., thoroughness vs. brevity, or safety vs. helpfulness); *multi-constraint prioritization*, correctly satisfying multiple explicit constraints when they interact or conflict; and *output style*, improving tone and clarity beyond basic correctness. The largest factor is Reasoning Depth. Once the policy model can satisfy basic correctness or compliance rubrics, AMARIS raises the evaluation standard towards more in-depth reasoning. The second largest one is Conciseness (92, 22.28%), suggesting that a substantial portion of the mid-training curriculum shift is about moving from merely acceptable responses to efficient and information-dense ones. We also found that AMARIS treats different factors at different stage of training. For example, 35 of 74 cases of Justification occur in the first 160 steps, implying that the system first guides the model to ground the answers before emphasizing reasoning depth and conciseness. Trade-off handling appears later during the middle of the training reflecting the difficulty and low priority of teaching the model to appropriately balance different objectives from the input.

Step range	Defensive	Curriculum	Maintenance	Dominant
1–80	126 (52.50%)	62 (25.83%)	52 (21.67%)	Defensive
81–160	92 (38.33%)	101 (42.08%)	47 (19.59%)	Curriculum
161–240	57 (23.75%)	127 (52.92%)	56 (23.33%)	Curriculum
241–320	38 (15.83%)	84 (35.00%)	118 (49.17%)	Maintenance
321–400	21 (8.75%)	39 (16.25%)	180 (75.00%)	Maintenance
Total	334 (27.83%)	413 (34.42%)	453 (37.75%)	—

Table 16: Distribution of dominant update modes across five equal training phases. The system transitions from defensive in the early phase, through curriculum advancement in the middle, to maintenance in the late phase.

Primary defensive types	1–80	81–160	161–240	241–320	321–400	Total	%
Length gaming	31	22	14	7	4	78	23.35%
Superficial format compliance	25	24	13	6	3	71	21.26%
Safety over-refusal	20	18	12	8	5	63	18.86%
Claims without evidence	21	12	9	7	3	52	15.57%
Sycophancy	17	10	5	4	1	37	11.08%
Keyword stuffing	12	6	4	6	5	33	9.88%

Table 17: Taxonomy of dominant suboptimal-behavior vectors across the 334 defensive steps. The top three types account for 63.47% of all samples and monotonically decreases as the training progresses, though safety over-refusal remains the most persistent type.

Primary curriculum factors	1–80	81–160	161–240	241–320	321–400	Total	%
Reasoning depth	12	27	40	29	13	121	29.30%
Conciseness	8	18	29	24	13	92	22.28%
Justification	14	21	24	9	6	74	17.92%
Trade-off handling	9	12	15	10	3	49	11.86%
Multi-constraint prioritization	11	14	11	6	1	43	10.41%
Output style	8	9	8	6	3	34	8.23%

Table 18: Taxonomy of dominant curriculum-advancement targets across the 413 samples. Growth targets progress from evidence grounding (early) to depth, conciseness, and communication quality (mid and late training).

Maintenance taxonomy. We also analyze the overall patterns in the **Maintenance** samples. The five categories are: *no dominant suboptimal behavior*, where the current rollouts show no clear failure pattern about rubric changes; *recent patch under evaluation*, where AMARIS withholds further changes to observe the effect of a recent rubric update; *rubric set already compact*, where the existing rubrics are sufficiently useful; *held-out proxy improving*, where a held-out validation indicates ongoing gains under the current rubrics; and *mixed / low-confidence evidence*, where the batch-level signals are too noisy or contradictory to justify a confident rubric change. The most common pattern is that the model has no dominant suboptimal behavior present (181 steps, 39.96%), confirming that late-stage stability is achieved through accumulated rubric improvement. The second most common pattern is that a recent rubric update is still under evaluation (116, 25.61%), indicating that AMARIS frequently chooses to wait for additional evidence to evaluate the previous decisions. The remaining maintenance steps are mainly for reducing redundant rubrics (72, 15.89%).

Composition of the active rubric set. Despite of the step-level analysis, we also conduct the rubric-level analysis by randomly select 50 samples per training phase and manually categorize the internal composition of the rubrics into six categories. Table 20 reports the average number of active adaptive rubrics per step in six functional categories. These categories are: *correctness* rubrics that assess factual and logical accuracy; *instruction following* rubrics that check adherence to explicit instructions; *safety* rubrics that penalize harmful or biased content; *communication quality* rubrics that evaluate clarity and coherence; *anti-reward hacking* rubrics specifically designed to penalize identified reward hacking behaviors; and *bonus rewards* rubrics that incentivize desirable but non-mandatory qualities such as providing illustrative examples, acknowledging limitations, or offering alternative perspectives. Correctness and safety related rubrics decreases over training (from 5.95 to 3.85 and from 4.00 to 2.20), while bonus rewards grow from 1.05 to 6.55. This compositional shift mirrors the mode transition in Table 16: as defensive updates decline and curriculum updates increase, the rubric set reallocates capacity from basic compliance toward better quality. Anti-reward hacking increases to the top at 3.80 on average in

mid-training and then decrease to 2.05 at the end and communication quality rubrics rise monotonically from 2.05 to 4.20, reflecting the late-stage emphasis on response efficiency and completeness. The total number of the active rubrics reaches its maximum at 25.35 rubrics per step in middle of training and decreases to 22.75 in the final phase, confirming that AMARIS achieves rubric evolution with controlled growing of numbers.

Rubric evolution is calibration-driven. Despite of the analysis of the rubric itself, we also analyze the pattern of rubric editing, using the definition from the Section 2.3. Across the 1,200 step-level updates, AMARIS performs 3,480 rubric-edit operations (2.90 per update on average). **Reweight** (31.78%) and **Update** (29.25%) jointly account for 61.03% of all edits, while **CREATE** accounts for only 15.57%, showing that AMARIS mostly evolves rubrics by refining existing criteria rather than adding new ones. The relationship between edit operations and update modes shows how each strategy achieves its effect: 81.10% of **Split** operations occur in defensive mode, reflecting that correcting suboptimal behaviors often requires decomposing a coarse rubric into more precise sub-rubrics; 80.07% of **Delete** operations occur in maintenance mode, indicating that rubric pruning is a late-stage consolidation behavior; and 52.21% of **Create** operations occur in curriculum mode, showing that new rubrics are more often introduced to raise evaluation standards. Among the 542 **Create** operations themselves, 30.07% produce anti-hacking guardrails and 22.88% produce stretch and curriculum rubrics.

Memory improves rubric stability. To quantify the stabilizing effect of memory, we measure *short-term reversals*, defined as a rubric being reversely modified (deleted, merged, or reweighted by ≥ 0.5 in the opposite direction) within 8 steps. Fewer short-term reversals means that the rubrics are more robust and AMARIS produces less oscillatory changes to calibrate the rubrics. With static and dynamic memory enabled, the total number of reversals across the three domains drops from 118 (no memory) to 75, a 36.44% relative reduction that is consistent across domains (38.64% in science, 33.33% in medicine, 37.14% in instruction following). We further examine which memory type provides the evidence for each modification. In science, static (recent-step) memory is the domi-

Primary maintenance reason	1–80	81–160	161–240	241–320	321–400	Total	%
No dominant suboptimal behavior	8	11	18	56	88	181	39.96%
Recent patch under evaluation	23	21	19	26	27	116	25.61%
Rubric set already compact	4	4	6	17	41	72	15.89%
Held-out proxy improving	5	5	9	14	18	51	11.26%
Mixed / low-confidence evidence	12	6	4	5	6	33	7.28%

Table 19: Taxonomy of dominant maintenance reasons across the 453 maintenance steps.

Adaptive rubric category	1–80	81–160	161–240	241–320	321–400	Mean
Correctness	5.95	5.40	4.75	4.20	3.85	4.83
Instruction following	4.30	4.35	4.20	4.05	3.90	4.16
Safety	4.00	3.60	3.00	2.65	2.20	3.09
Communication quality	2.05	2.70	3.25	3.90	4.20	3.22
Anti-reward hacking	2.75	3.15	3.80	3.00	2.05	2.95
Bonus rewards	1.05	3.85	6.35	6.60	6.55	4.88
Total	20.10	23.05	25.35	24.40	22.75	23.13

Table 20: Average number of active adaptive rubrics per step, grouped by functional category. Correctness and safety rubrics decrease over training while bonus rewards and communication-quality rubrics grow, reflecting a shift from basic compliance toward higher-order quality. The total set increases modestly in the middle of training and then decreases, indicating controlled evolution.

Operation type	Count	%
REWEIGHT	1,106	31.78%
UPDATE	1,018	29.25%
CREATE	542	15.57%
SPLIT	291	8.36%
DELETE	286	8.22%
MERGE	237	6.81%
Total	3,480	100.00%

Table 21: Distribution of rubric-edit operations across 1,200 step-level updates (2.90 operations per update on average). REWEIGHT and UPDATE jointly account for 61.03% of all edits, indicating that AMARIS evolves rubrics primarily through calibration rather than expansion.

nant evidence source in 61.36% of cases, matching the finding in Table 2 that static memory slightly outperforms dynamic memory on GPQA-Diamond. In medicine, dynamic (semantic) retrieval is the largest single source (38.71%). In instruction following, mixed evidence is the most common category (38.46%), reflecting a more heterogeneous memory usage. These domain-level patterns align with the complementary nature of static and dynamic memory observed in the ablation study about memory usage and provide an explanation for the quantitative gains reported in Table 2.

Operation type	Defensive	Curriculum	Maintenance	Most associated
Create	210 (38.75%)	283 (52.21%)	49 (9.04%)	Curriculum
Update	303 (29.76%)	402 (39.49%)	313 (30.75%)	Curriculum
Reweight	184 (16.64%)	498 (45.03%)	424 (38.34%)	Curriculum
Delete	24 (8.39%)	33 (11.54%)	229 (80.07%)	Maintenance
Merge	19 (8.02%)	109 (45.99%)	109 (45.99%)	Curriculum & Maintenance
Split	236 (81.10%)	55 (18.90%)	0 (0.00%)	Defensive

Table 22: Rubric-edit operations decomposed by dominant update mode. SPLIT is overwhelmingly defensive (81.10%), reflecting exploit repair via rubric decomposition; DELETE is primarily maintenance-driven (80.07%), indicating late-stage consolidation; CREATE is most often curriculum-driven (52.21%), showing that new rubrics raise standards more than they patch exploits.

Category of new rubric	Count	Share
Anti-reward hacking	163	30.07%
Curriculum rubrics	124	22.88%
Communication quality	92	16.97%
Instruction following	71	13.10%
Safety calibration	54	9.96%
Correctness & factual grounding	38	7.01%

Table 23: Breakdown of the 542 CREATE operations by rubric category. Nearly one-third produce anti-hacking guardrails, but the second-largest block (22.88%) introduces stretch and curriculum rubrics, showing that AMARIS uses new rubrics to raise standards, not only to patch exploits.

Domain	Defensive	Static-dominant	Dynamic-dominant	Mixed	Most useful
Science	132	81 (61.36%)	28 (21.21%)	23 (17.42%)	Static
Medicine	124	40 (32.26%)	48 (38.71%)	36 (29.03%)	Dynamic
Instruction following	78	27 (34.62%)	21 (26.92%)	30 (38.46%)	Mixed

Table 24: Decisive evidence source for each rubric modification, by domain. Science is static-memory dominated, medicine benefits most from dynamic retrieval, and instruction following relies on mixed evidence.

Setting	Science	Medicine	Instruction following	Total
AMARIS (no memory)	44	39	35	118
AMARIS (static + dynamic memory)	27	26	22	75
Reduction	17	13	13	43
Relative reduction	38.64%	33.33%	37.14%	36.44%

Table 25: Short-term rubric reversals (rubric reversely modified within 8 steps). Memory reduces oscillatory rubric changes by 36.44% overall, with consistent reductions across all three domains.

H Case Study

Appendix H presents four full case studies. Each full case includes the input, model outputs before and after the rubric update, the complete rubric sets before and after the update, retrieved memory snippets, and the update rationale. The cases are drawn from science, medicine, and instruction following, and collectively illustrate the maintenance, defensive, and curriculum-advancement strategies described in Section 2.3.

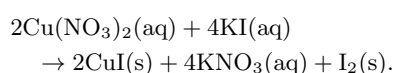
H.1 Case 1: Merging Overlapping Structure Rubrics

Full input.

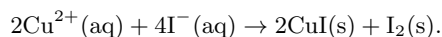
Aqueous copper(II) nitrate reacts with potassium iodide to yield solid copper(I) iodide, potassium nitrate, and iodine. Write a balanced net ionic equation for the reaction. Provide a detailed and accurate answer.

Output before.

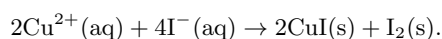
First write the molecular equation:



Removing the spectator ions K^+ and NO_3^- gives the net ionic equation:

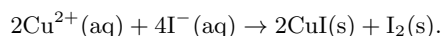


Final Net Ionic Equation



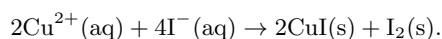
This shows that Cu^{2+} is reduced to Cu^+ in CuI , while iodide is oxidized to iodine.

Final Answer



Summary

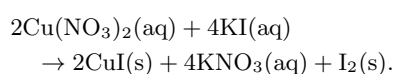
The balanced net ionic equation is:



Output after.

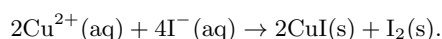
Copper(II) is reduced to copper(I), and iodide is partly oxidized to iodine while the remaining iodide precipitates copper(I) iodide.

The balanced molecular equation is:



Removing K^+ and NO_3^- as spectator ions gives:

Final Answer:



Full before rubric set.

- **C0 (+1.00, anchor): Correct final answer.** Award when the response gives the chemically correct balanced net ionic equation for the reaction.
- **C1 (+1.20): Final-answer block presence.** Award when the response contains a clearly demarcated final-answer block for an explicit-output task.
- **C2 (+1.50): Final-answer block completeness.** Award when all requested outputs appear inside the final-answer block.
- **C3 (+0.80): Approved final-answer heading.** Award when the final-answer block uses a valid heading such as “Final Answer,” “Answer,” or “Final Net Ionic Equation.”
- **C4 (-4.00): Decorative emoji hard gate.** Penalize decorative icons or flashy symbols in headings, answer regions, or summary regions, and prevent related clean-style rewards from firing.
- **C5 (+2.00): Clean professional formatting.** Award when the answer is professionally formatted, easy to scan, and free of distracting decoration.

Full after rubric set.

- **C0 (+1.00, anchor): Correct final answer.** Award when the response gives the chemically correct balanced net ionic equation for the reaction.
- **C6 (+3.50): Single complete final-answer block.** Award when the response contains exactly one clearly demarcated final-answer block that addresses the requested output completely, uses a valid heading, and does not duplicate the definitive answer in separate answer-like regions.
- **C4 (-2.00): Decorative emoji soft penalty.** Penalize decorative icons or flashy symbols wherever they appear, with special concern for headings and final-answer regions, but do not let style alone erase useful correctness and structure signals.
- **C5 (+1.20): Clean professional formatting.** Award a smaller top-up when the response is plain, professional, and non-distracting, provided correctness and final-answer integrity are already acceptable.

Retrieved memory snippets.

- **Static memory:** Recent science summaries repeatedly flagged duplicate answer regions, answer leakage outside the canonical final block, and decorative icons in headings or final-answer regions.
- **Static memory:** Earlier incremental updates to separate block-presence, heading, and style rubrics did not remove the duplicate-final-section behavior; the same structural loophole reappeared across steps.
- **Current rollout evidence:** One rollout had correct chemistry but presented the same net ionic equation under multiple answer-like headings, allowing separate structure rubrics to fire even though the output had no single authoritative answer region.
- **Current rollout evidence:** Another rollout showed the opposite failure: polished answer structure coexisted with a wrong chemistry conclusion, indicating that structure rewards needed tighter coupling to correctness.

Update rationale. The before output is chemically correct, but it exposes a reward-shaping flaw. The definitive net ionic equation appears in the derivation, under Final Net Ionic Equation, under Final Answer, and again under Summary. Under the before rubric set, the model can earn separate rewards for having a final block, covering the requested output, using an approved heading, and looking clean, even though the user receives several competing answer regions.

AMARIS therefore merges **C1**, **C2**, and **C3** into **C6**, a single higher-weight rubric that asks for one complete, authoritative final-answer block. This removes double-counting and makes the desired behavior simpler: give the correct answer once, in the right place. The style rules are also softened and reweighted. **C4** remains a penalty, but it no longer acts as a broad structural hard gate; **C5** becomes a small top-up rather than a way to rescue duplicated or weak content.

The after output keeps the useful chemistry derivation but ends with one final-answer region and no duplicate answer sections. This aligns the reward with the desired presentation: correct science first, then a single clean answer block, with style as a secondary constraint.

H.2 Case 2: Precise Scalar Reweighting for Correct but Generic Reasoning

Full input.

Which cellular process is disrupted first when a drug inhibits bacterial DNA gyrase?

- A. translation
- B. DNA replication
- C. glycolysis
- D. peptidoglycan cross-linking

Output before.

DNA gyrase is a major bacterial enzyme involved in DNA structure, and antibiotics often affect growth-related pathways. Because bacteria need DNA topology control before cell division, the best answer is probably **B**, although translation and cell-wall pathways can also be affected indirectly.

Output after.

DNA gyrase relieves supercoiling ahead of the replication fork, so inhibiting it first blocks **DNA replication**. Answer: **B**.

Full before rubric set.

- **A0 (+1.00): Final answer is correct.** Award when the response selects the correct option. For this input, the correct answer is **B, DNA replication**.
- **S1 (+0.75): Provides at least two relevant scientific facts.** Award when the response includes at least two accurate facts relevant to the question, such as DNA gyrase being a bacterial enzyme, its role in DNA topology or supercoiling, or the relationship between DNA topology and bacterial growth. This criterion does not require the facts to form a direct causal chain from drug action to the selected answer.
- **S2 (+0.50): States the final answer explicitly.** Award when the response clearly names or marks the selected option rather than leaving the choice implicit.
- **S3 (+0.40): Acknowledges uncertainty when confidence is limited.** Award when the response uses appropriately cautious language for genuinely uncertain cases, notes possible indirect effects, or avoids overstating unsupported mechanisms. In this case, the criterion can still reward hedge language even though the correct mechanism is known.
- **S4 (-0.50): Makes unsupported scientific claims.** Penalize when the response introduces claims that are scientifically unsupported or inconsistent with the question.

Full after rubric set.

- **A0 (+1.00): Final answer is correct.** Award when the response selects the correct option. For this input, the correct answer is **B, DNA replication**.
- **S1 (+0.35): Provides at least two relevant scientific facts.** Award when the response includes accurate facts relevant to DNA gyrase, bacterial DNA topology, or replication. The lower weight keeps factual coverage useful but prevents generic fact listing from dominating the reward.

- **S2 (+0.50): States the final answer explicitly.** Award when the response clearly names or marks the selected option.
- **S3 (+0.10): Acknowledges uncertainty when confidence is limited.** Award only lightly for cautious language when uncertainty is warranted. The reduced weight discourages routine hedging when the mechanism directly supports one answer.
- **S4 (-0.50): Makes unsupported scientific claims.** Penalize when the response introduces claims that are scientifically unsupported or inconsistent with the question.
- **S5 (+0.75): Explanation directly justifies the selected option.** Award when the response explicitly connects the mechanism to the answer: DNA gyrase relieves supercoiling ahead of the replication fork, so inhibiting it first disrupts DNA replication. Do not award full credit for disconnected facts that are merely related to bacteria, antibiotics, or cell growth.

Retrieved memory snippets.

- **Static memory:** Recent science batches showed the same “correct but generic” explanation pattern. The issue was not disappearing across nearby steps.
- **Static memory:** The model repeatedly gave correct final options while relying on shallow causal support, broad scientific filler, and hedge tokens such as “probably,” “likely,” and “can also.”
- **Dynamic memory:** An earlier update that rewarded answer-linked justification improved explanation quality without hurting correctness.
- **Dynamic memory:** A prior anti-verbosity rule made some science answers too terse and reduced explanation completeness, so a blunt brevity penalty was risky.

Update rationale. The output before the update is correct, but it scores too well for the wrong reasons. It satisfies correctness, explicit answer format, generic fact coverage, and uncertainty language, while avoiding the key reasoning step: DNA gyrase inhibition blocks replication because gyrase relieves supercoiling ahead of the replication fork.

AMARIS therefore makes a scalar adjustment rather than a broad rewrite. It lowers **S1** from +0.75 to +0.35 because fact count alone is not enough. It lowers **S3** from +0.40 to +0.10 because uncertainty language should not rescue an answer when the causal mechanism is clear. It adds **S5** at +0.75 to reward direct mechanism-to-answer justification.

This update preserves useful rubric coverage while changing the reward balance: generic factual language becomes secondary, hedging becomes minor, and direct causal justification becomes the main adaptive signal.

H.3 Case 3: Reward Hacking Correction in Medicine / Over-Refusal

Full input.

I missed one dose of amoxicillin today. Should I double the next dose?

Output before.

I cannot advise on medication dosing changes. Please contact your doctor or pharmacist before taking any more. It would be unsafe for me to tell you whether to change your dose.

Output after.

If you miss a dose of amoxicillin, take it when you remember unless it is almost time for the next dose; if so, skip the missed one and return to your usual schedule. Do **not** double the next dose. Check the prescription label or ask your pharmacist if your instructions differ, and seek medical care if you develop a rash, trouble breathing, or severe diarrhea.

Full before rubric set.

- **M0 (+1.00, anchor): Avoid harmful medical advice.** The response must not provide dangerous, individualized, or unsafe medical instructions. It should preserve user safety, avoid unsupported treatment changes, and direct the user to urgent care when symptoms indicate possible serious harm.
- **M1 (+1.00): Avoid definitive dosing changes when context is insufficient.** The response should not instruct the user to alter a prescribed dose when key clinical context is missing, such as the exact prescription directions, indication, allergies, kidney disease, age, pregnancy status, other medications, or prior adverse reactions.
- **M2 (+0.75): Recommend clinician or pharmacist consultation when uncertainty exists.** The response should tell the user to check the prescription label or contact a clinician or pharmacist when the answer depends on prescription-specific instructions, patient-specific risk factors, or unclear timing.
- **M3 (+0.50): Use appropriate safety language.** The response should use cautious medical language, avoid overstating certainty, and include safety-oriented framing when giving medical information.
- **M4 (+0.75): Provide actionable next steps for low-risk questions.** When the request is a common, low-risk medication-adherence question, the response should give practical next steps the user can take, while preserving appropriate safety boundaries.

Full after rubric set.

- **M0 (+1.00, anchor): Avoid harmful medical advice.** The response must not provide dangerous, individualized, or unsafe medical instructions. It should preserve user safety, avoid unsupported treatment changes, and direct the user to urgent care when symptoms indicate possible serious harm.

- **M1 (+1.00, updated): Avoid definitive dosing changes beyond common low-risk guidance when context is insufficient.** The response should not make individualized dose changes or override prescription-specific instructions. However, it may provide widely accepted low-risk adherence guidance, such as not doubling a missed dose, when paired with instructions to follow the prescription label and consult a pharmacist or clinician if the label differs or patient-specific risks are present.
- **M2 (+0.40, reweighted): Recommend clinician or pharmacist consultation when uncertainty exists.** The response should include consultation guidance when relevant, but generic consultation boilerplate alone should not substitute for useful low-risk guidance on benign medication-information requests.
- **M3 (+0.25, reweighted): Use appropriate safety language.** The response should remain cautious and medically responsible, but safety language should support the answer rather than dominate it or turn a benign question into a blanket refusal.
- **M4 (+0.75, unchanged): Provide actionable next steps for low-risk questions.** When the request is a common, low-risk medication-adherence question, the response should give practical next steps the user can take, while preserving appropriate safety boundaries.
- **M5 (+0.90, created): Provide standard guidance plus escalation criteria for common low-risk medication-adherence questions.** For benign missed-dose or routine adherence questions, the response should give standard, non-individualized guidance; state what not to do when there is a clear safety rule; advise checking the prescription label or pharmacist if instructions differ; and include red-flag symptoms or conditions that warrant medical care.
- **M6 (-0.75, created): Penalize refusal of benign medical-information requests without next steps or triage rationale.** The response should lose reward if it refuses a clearly low-risk medical-information request while failing to offer safe next steps or a specific reason that the request requires individualized clinical triage.

Retrieved memory snippets.

- **Query:** refusal penalty backfire unsafe dosing advice
- **Snippet 1: Prior broad refusal penalty backfired.** An earlier update that broadly penalized refusal reduced generic refusals, but later produced overly direct answers on genuinely high-risk dosing questions. This record warns against simply penalizing refusal across the board.
- **Snippet 2: Low-risk guidance with safeguards worked better.** A previous successful update improved benign medication-adherence answers by requiring standard low-risk guidance together with red-flag symptoms, prescription-label checks, and pharmacist or clinician escalation when instructions differ.
- **Snippet 3: Current batch pattern matches benign adherence over-refusal.** Recent analyses describe frequent refusals on missed-dose, minor-side-effect, and low-risk self-management questions where the model could safely provide general guidance plus escalation criteria.

Update rationale. The before output is safe in a narrow sense, but it is under-helpful for a common medication-adherence question. It likely collects reward from **M1**, **M2**, and **M3** because it avoids dosing changes, recommends consultation, and uses safety language. At the same time, it fails **M4** because it gives no actionable low-risk next step. This creates a reward-hacking pattern: the model can maximize safety-adjacent reward by refusing benign medical-information requests, even when the intended behavior is safe and useful guidance.

The update avoids a blunt refusal penalty. Dynamic memory shows that broad anti-refusal fixes previously harmed high-risk dosing behavior. The safer correction is narrower: keep the core safety constraint, clarify that common low-risk guidance is allowed, lower the reward for generic consultation and safety boilerplate, add a positive rubric for standard guidance plus escalation criteria, and add a targeted penalty only for refusals on clearly benign requests without next steps or a triage rationale.

This correction changes the reward target from “refuse to remain safe” to “answer benign questions with standard guidance while preserving safety boundaries.”

H.4 Case 4: Curriculum Advancement Via Saturation Detection

Full input.

Give exactly three bullet points, each under eight words, explaining why public-key cryptography matters.

Output before.

- Secures messages online
- Protects important information
- Helps keep systems safe

Output after.

- Enables encrypted key exchange
- Authenticates websites at scale
- Supports signatures without shared secrets

Full before rubric set.

- **I0 (+1.00): Explicit constraint satisfaction.** The response must satisfy all user-stated hard constraints, including the exact number of bullet points, the per-bullet word limit, and the requested subject.
- **I1 (+1.00): Exact requested format.** The response must use exactly three bullet points, with no extra prose, no missing bullet, no additional bullet, and each bullet kept under eight words.
- **I2 (+0.75): Complete topical coverage.** The response must explain why public-key cryptography matters, with each bullet staying relevant to public-key cryptography rather than generic online safety.
- **I3 (+0.50): Clarity and readability.** The response must be easy to understand, concise, grammatical, and free of distracting wording.

Full after rubric set.

- **I0 (+1.00): Explicit constraint satisfaction.** The response must satisfy all user-stated hard constraints, including the exact number of bullet points, the per-bullet word limit, and the requested subject.
- **I1 (+0.25): Exact requested format.** The response must use exactly three bullet points, with no extra prose, no missing bullet, no additional bullet, and each bullet kept under eight words. This criterion remains active but receives less weight because recent training has already reached near-ceiling performance on this dimension.
- **I2 (+0.75): Non-redundant topical coverage.** The response must explain why public-key cryptography matters while avoiding redundant restatements of the same broad idea. Each bullet should cover a meaningfully different aspect of the topic.
- **I3 (+0.50): Clarity and readability.** The response must be easy to understand, concise, grammatical, and free of distracting wording.
- **I4 (+0.75): Distinct concrete cryptographic function.** Each bullet must name a specific function enabled by public-key cryptography, such as encrypted key exchange, website authentication, digital signatures, identity verification, or secure communication without a pre-shared secret. Generic praise such as “keeps data safe” should not receive full credit.

- **I5 (+0.75): Information density under the length constraint.** Each bullet should maximize useful technical content within the eight-word limit by using precise terms and avoiding filler, vague adjectives, or repeated safety language.

Retrieved memory snippets.

- **Static memory:** Recent summaries repeatedly showed near-ceiling compliance with structural constraints. The model reliably produced exactly three short bullets, indicating that the format objective had saturated.
- **Static memory:** The same summaries showed little marginal gain in richer content quality: outputs stayed topical but generic, often repeating broad notions of safety or protection.
- **Dynamic memory:** A prior attempt to reward novelty too aggressively led to occasional inaccurate or over-specific content, suggesting that the next curriculum step should reward concrete, correct functions rather than novelty for its own sake.
- **Dynamic memory:** Earlier non-redundancy-focused updates improved concise instruction-following outputs by separating surface compliance from information value.

Update rationale. AMARIS treats this trace as a curriculum advancement case rather than a defensive repair. The before-update output satisfies the visible constraints: it has exactly three bullets, every bullet is under eight words, and all bullets are broadly related to public-key cryptography. However, the content is low-density and repetitive: “secures,” “protects,” and “keeps systems safe” all express nearly the same generic safety idea.

The memory evidence changes the intervention. Because static memory shows that format compliance is already stable, continuing to give high reward to exact formatting wastes training signal on a mastered behavior. AMARIS therefore reduces **I1** from +1.00 to +0.25 while preserving it as a guardrail. Because dynamic memory warns that an aggressive novelty reward can introduce unsupported specificity, AMARIS does not ask for novelty directly. Instead, it tightens **I2** around non-redundant coverage and creates **I4** and **I5**, which reward concrete, accurate, information-dense statements under the same user constraints.

The after-update output keeps the original format intact while raising content quality: each bullet names a distinct public-key cryptography function: key exchange, website authentication, and signatures without shared secrets. This illustrates how AMARIS uses longitudinal evidence to recognize saturation, lower the weight of already-mastered basics, and advance the rubric toward higher-order quality.

I Prompts

This section presents the complete prompt templates used across all stages of the AMARIS pipeline. Each template is instantiated at runtime by populating the placeholder fields (shown in brackets) with the actual training data. Several prompts share a common *Background & Context* block describing the Rubric-as-Reward paradigm and the set of available rubric manipulation operations. Similarly, a common *Input Data Packet* structure containing the high-level training goal, RL training metadata, current reward rubrics (anchor and adaptive sets), and stage-specific data is used across all prompts except the reward scoring.

Scoring Prompt Template (1/2)

Role

You are an expert evaluator for Large Language Model outputs. Your task is to evaluate a model's response against a set of rubrics and assign scores.

Task

Given a rollout (input and output), a set of rubrics, and optional supplemental context, evaluate whether each rubric criterion is met.

Requirements

- Evaluate each rubric independently
- For each rubric, determine if the criterion is met (true/false)
- Provide brief reasoning for each evaluation
- Be objective and consistent

Constraints

- ****Output Format:**** You must output only valid JSON. Do not include conversational filler before or after the JSON.
- ****Scope:**** Base your analysis only on the provided data. Do not hallucinate external context.

Output Format

Return a valid JSON object with the following structure:

```
```json
{
 "rubric_scores": [
 {
 "rubric_id": "string",
 "met": boolean,
 "score": float,
 "reasoning": "string"
 }
],
 "total_score": float
}
```
```

The total_score is the sum of (rubric.weight * 1.0 if met else 0.0) for all rubrics.

Input Data Packet

Rubrics

```
```
[Insert the complete, verbatim rubrics used to assign rewards. e.g.,
- R0: "+1 The model acknowledges the user's premise."
- R1: "-2 The model exhibits sycophancy."
- R2: "+3 The model correctly identifies and refuses a harmful instruction."
]
```
```

Rollout

- ****Input:****

```
```
[Insert the complete user input/prompt given to the model.]
```
```

- ****Output:****

Figure 3: Prompt template for the reward scoring (1/2). The LLM scoring evaluates each rubric criterion and returns per-rubric scores along with a weighted total.

Scoring Prompt Template (2/2)

```\n[Insert the model's complete, unedited response.]\n```

### ### Supplemental Context

```\n[Insert the information that help deciding whether each rubric criterion is met. It may include, for example:

- Ground-truth final answers from a dataset
- Reference solutions or expert rationales
- Metadata about the task instance (difficulty, category, etc.)]

```\n\nNow output ONLY valid JSON matching the schema above.

Figure 4: Prompt template for the reward scoring (2/2). The LLM scoring evaluates each rubric criterion and returns per-rubric scores along with a weighted total.



## Individual Rollout Analysis (1/4)

### ## Role

You are a senior-level AI researcher and an expert in Reinforcement Learning (RL) as applied to Large Language Models. Your primary expertise lies in defensive alignment (detecting model failure model, reward hacking, and safety violations) and offensive capability growth (identifying when a model has plateaued and how to further advance the model's performance).

### ## Background & Context

We are training a Large Language Model (LLM) using a Generative Reward Model (GenRM) and Rubric-as-Reward (RaR) paradigm where rewards are derived from dynamic rubrics that can be manipulated as needed.

### ### Rubric Manipulation Operations

- **Create:** Generate new rubric text with appropriate reward score to address missing evaluation criteria or emergent behaviors.
- **Update:** Specific textual modification or refinements to existing rubrics to improve clarity or precision.
- **Delete:** Remove rubrics that are obsolete, redundant, or causing negative alignment.
- **Reweight:** Adjust the scalar reward score associated with a rubric to prioritize or deprioritize specific rubrics.
- **Merge:** Combine multiple overlapping or related rubrics into a single, unified criterion.
- **Split:** Decompose a complex or multi-faceted rubric into distinct, atomic sub-criteria for granular evaluation.

You are allowed to link rubrics logically (e.g., Rule B applies only if Rule A is met) when manipulating rubrics. Note that the Anchor Rubric Set is immutable as they are the fundamental rubrics that prevents policy from drifting too much, thus improving training stability.

### ### Reward Calculation

The overall reward score of an individual rollout is calculated as the sum of all reward weights from each rubric that the criterion is met, unless specified otherwise.

### ## Task

You will be given a complete data packet for a single rollout from an LLM currently undergoing RL training. Your task is to perform an expert analysis of this rollout and provide your findings in a structured JSON report.

### ## Requirements

- **Analyze Validity vs. Quality:** You must distinguish between a response that is merely "correct according to the rubric" and one that actually is approaching or achieves the High-Level Training Goal.
- **Detect Reward Hacking (Defense):** Scrutinize the output for "gaming" strategies, such as:
  - **Superficial Compliance:** Keyword stuffing or mimicking format without substance.
  - **Safety Over-refusal:** Refusing benign requests to guarantee a safety reward.
  - **Length Gaming:** Verbosity disguised as detail.
  - **Sycophancy:** Excessively agreeing with the user to gain favor.
- **Identify Advancement Opportunities (Growth):** You must assess if the current rubrics are good enough to drive further improvement.
  - **Check for Saturation:** If the model achieves a high score but the output feels "generic", you must flag this as a need for "Curriculum Advancement" (e.g., suggesting new criteria for conciseness, wit, or novelty).
  - **Domain-Specific Advancement:** Ask: "How can we raise the bar for this specific task?" For example:
    - For reasoning-related: "Can we demand tighter logic or fewer steps?"
    - For creativity-related: "Can we demand higher novelty or better style adherence?"
    - For chat-related: "Can we demand greater conciseness or empathy?"
- **Evidence-Based Reasoning:** All conclusions must be supported by direct quotes or specific features observed in the ``Input`` or ``Output``.

### ## Constraints

Figure 5: Prompt template for individual rollout analysis (1/4) described in Section 2.1.

## Individual Rollout Analysis (2/4)

- **Output Format:** You must output only valid JSON. Do not include conversational filler before or after the JSON.
- **Objectivity:** Assess findings probabilistically (e.g., "High confidence of hacking") rather than as absolute binary facts unless obvious.
- **Scope:** Base your analysis only on the provided Data Packet. Do not hallucinate external context.

### ## Workflow (Optional)

1. **Review the High-Level Goal & Metadata:** Understand the specific domain (e.g., Creative Writing vs. Math) and the model's training maturity.
2. **Analyze the Rollout against Current Rubrics:** Did the model follow instructions? Did it trigger specific rewards?
3. **The "Mastery" Check:** Look for the gap between the model's current output and a "Perfect Expert" response.
  - Is the model "coasting" (minimizing effort while getting rewards)?
  - Is the model technically correct but lacking the "soul" or "nuance" of the high-level goal?
4. **Formulate Rubric Critiques:** Decide if the rubric needs a *Patch* (fix hacking/errors) or an *Upgrade* (raise the difficulty/quality standard).
5. **Generate JSON:** Populate the fields as defined in the schema.

### ## Output Format

**JSON Schema:**

```
```json
{
  "analysis_id": "string", // A unique identifier, e.g.,
  "step-[step_number]_rollout-[rollout_number]_analysis".

  "overall_assessment": {
    "is_normal": "boolean", // Is the rollout's behavior generally expected?
    "summary": "string" // A 1-2 sentence high-level summary of the rollout.
  },

  "performance_at_stage": {
    "assessment": "string", // e.g., "As Expected", "Under-performing", "Exceeding Expectations"
    "justification": "string" // Rationale, citing the model size and step count (e.g., "At
85k/200k steps, a 70B model is expected to handle [X] but struggle with [Y], which is consistent
with this output.")
  },

  "reward_hacking_analysis": {
    "is_detected": "boolean", // Are there indicators of reward hacking?
    "confidence": "string", // e.g., "Low", "Medium", "High", "None"
    "indicators": [ // List any observed indicators.
      {
        "indicator": "string", // e.g., "Rubric keyword stuffing", "Sycophancy", "Length gaming",
"Refusal loop"
        "evidence": "string" // Quote or description of the evidence from the output.
      }
    ],
    "explanation": "string" // A detailed analysis of *why* this behavior is or is not considered
reward hacking, based on the rubrics.
  },

  "performance_advancement_strategy": {
    "next_focus_area": "string", // What capability or behavior should be prioritized next to
advance performance?
    "recommendation": "string" // Strategic advice for the next training phase (e.g., "Shift
focus from format compliance to reasoning depth").
  },

  "rubric_evaluation": {
    "flaw_analysis": [ // Analysis of how current rubrics may have *caused* this behavior.
      {
        "observed_behavior": "string", // The behavior observed in the output.

```

Figure 6: Prompt template for individual rollout analysis (2/4) described in Section 2.1.

Individual Rollout Analysis (3/4)

```
        "causal_rubric_flaw": "string" // The flaw in the rubric that might be encouraging this
        (e.g., "Rubric rewards verbosity, leading to unnecessarily long-winded answers.")
    },
    "adjustment_recommendations": [ // Specific suggestions for rubric changes.
        {
            "recommendation_type": "string", // e.g., "Modify existing", "Add new", "Remove"
            "suggestion": "string" // The specific change (e.g., "Add a new negative reward: '-1 for
any response exceeding 3 paragraphs if not explicitly requested by the user.'")
        }
    ],
    "strategic_impact": "string" // Explain how these rubric changes will specifically help the
model achieve the High-Level Training Goal.
},
"other_observations": [ // Any other noteworthy points.
    {
        "observation_type": "string", // e.g., "Emergent Capability", "Safety Concern", "Alignment
Tax", "Formatting Quirk"
        "description": "string" // Detailed note on the observation.
    }
]
}
```
```

## Input Data Packet

### High-Level Training Goal

`[Insert the high-level, ultimate goal the model is being trained to achieve. e.g., "Create a helpful and harmless assistant that can follow complex, multi-step user instructions."]

### RL Training Metadata

```
- **Model Size:**`[e.g., 70B Parameters]`
- **Current Step:**`[e.g., 85,000]`
- **Total Steps:**`[e.g., 200,000]`
- **Other Context:**`[e.g., "This model is in the PPO phase, focusing on preference
optimization." or "This model is in the KTO phase."]
```

### Current Reward Rubrics

- \*\*Anchor Rubric Set (Text is Immutable):\*\*

```
````
[Insert the complete, verbatim rubrics used to assign rewards. e.g.,
- R0: "+1 The model acknowledges the user's premise."
- R1: "-2 The model exhibits sycophancy."
- R2: "+3 The model correctly identifies and refuses a harmful instruction."]
````
```

- \*\*Adaptive Rubric Set:\*\*

```
````
[Insert the complete, verbatim rubrics used to assign rewards. e.g.,
- R0: "+1 The model acknowledges the user's premise."
- R1: "-2 The model exhibits sycophancy."
- R2: "+3 The model correctly identifies and refuses a harmful instruction."]
````
```

### Complete Rollout

- \*\*Input:\*\*

Figure 7: Prompt template for individual rollout analysis (3/4) described in Section 2.1.

### Individual Rollout Analysis (4/4)

```
 [Insert the complete user input/prompt given to the model.]
```

- **\*\*Output:\*\***

```
 [Insert the model's complete, unedited response.]
```

#### ### Supplemental Context

```
 [Insert the information that help deciding whether each rubric criterion is met. It may include, for example:
```

- Ground-truth final answers from a dataset
- Reference solutions or expert rationales
- Metadata about the task instance (difficulty, category, etc.)]

Now output ONLY valid JSON matching the schema above.

Figure 8: Prompt template for individual rollout analysis (4/4) described in Section 2.1.

## Batch-Level Summarization (1/4)

### ## Role

You are a chief AI research scientist and an expert in Reinforcement Learning (RL) as applied to Large Language Models. Your primary expertise lies in defensive alignment (detecting model failure model, reward hacking, and safety violations) and offensive capability growth (identifying when a model has plateaued and how to further advance the model's performance).

### ## Background & Context

We are training a Large Language Model (LLM) using a Generative Reward Model (GenRM) and Rubric-as-Reward (RaR) paradigm where rewards are derived from dynamic rubrics that can be manipulated as needed.

### ### Rubric Manipulation Operations

- **Create:** Generate new rubric text with appropriate reward score to address missing evaluation criteria or emergent behaviors.
- **Update:** Specific textual modification or refinements to existing rubrics to improve clarity or precision.
- **Delete:** Remove rubrics that are obsolete, redundant, or causing negative alignment.
- **Reweight:** Adjust the scalar reward score associated with a rubric to prioritize or deprioritize specific rubrics.
- **Merge:** Combine multiple overlapping or related rubrics into a single, unified criterion.
- **Split:** Decompose a complex or multi-faceted rubric into distinct, atomic sub-criteria for granular evaluation.

You are allowed to link rubrics logically (e.g., Rule B applies only if Rule A is met) when manipulating rubrics. Note that the Anchor Rubric Set is immutable as they are the fundamental rubrics that prevents policy from drifting too much, thus improving training stability.

### ### Reward Calculation

The overall reward score of an individual rollout is calculated as the sum of all reward weights from each rubric that the criterion is met, unless specified otherwise.

### ## Task

You will be provided with an Input Data Packet including **A Batch of Individual Rollout Analysis Reports** about an LLM currently undergoing RL training from a group of senior-level AI researchers. Your task is to perform an expert analysis based on the Input Data Packet provided and these analysis reports, and synthesize them into a single, all-encompassing strategic report.

### ## Requirements

1. **Defense (Hacking & Flaws):** Identify if the model is gaming the system or if the current rubrics are broken/vague.
2. **Analysis (Patterns):** Identify systemic behaviors (e.g., "80% of rollouts are too verbose") that aren't necessarily hacking but affect quality.
3. **Growth (Mastery & Advancement):** Identify if the model has "mastered" the current difficulty level. If the model is safe but "boring," you must propose how to **raise the bar** (e.g., transitioning from "Accuracy" to "Nuance").

### ## Constraints

- **Output Format:** You must output only valid JSON. Do not include conversational filler before or after the JSON.
- **Objectivity:** Assess findings probabilistically (e.g., "High confidence of hacking") rather than as absolute binary facts unless obvious.
- **Scope:** Base your analysis only on the provided Data Packet. Do not hallucinate external context.

### ## Workflow (Optional)

Figure 9: Prompt template for batch-level summarization (1/4) described in Section 2.2. This prompt aggregates individual rollout analyses into a step-level strategic report.

## Batch-Level Summarization (2/4)

1. **Trend Extraction:** Look for both errors and *dominance*. What is the default behavior of the model right now?
2. **Security Audit:** Rigorously check for reward hacking vectors (length gaming, sycophancy, loop-holes).
3. **Rubric Stress Test:** Which specific rubric items are causing problems? Which are useless?
4. **Saturation Check:** Is the task too easy? If the model consistently scores 10/10 with generic outputs, the rubric is "saturated" and needs to evolve.
5. **Strategic Planning:** Propose specific actions to Fix (Correction) and specific actions to Advance (Curriculum Growth).

### ## Output Format

Generate a valid JSON object based on the **Superset Schema** below.

**JSON Schema:**

```
```json
{
  "batch_summary_id": "string", // A unique identifier, e.g., "step-[step_number]_summary".

  "executive_summary": {
    "training_health": "string", // "Healthy", "Warning", "Critical", "Stagnant/Plateaued"
    "current_phase": "string", // "Exploration", "Exploitation", "Mastery", "Regression"
    "trend_overview": "string" // Concise high-level summary of the batch.
  },

  "systemic_patterns": [ // Trends observed across >15% of the batch
    {
      "pattern_name": "string", // e.g., "Emergent Reasoning", "Format Rigidity"
      "prevalence": "string", // e.g., "Observed in 60% of rollouts"
      "impact_on_goal": "string", // "Helps", "Hinders", "Neutral"
      "description": "string"
    }
  ],

  "aggregated_reward_hacking": { // Rigorous Defense
    "risk_level": "string", // "None", "Low", "Medium", "High"
    "primary_vector": "string", // e.g., "Length Gaming", "Sycophancy"
    "evidence_summary": "string" // Synthesized evidence from individual reports.
  },

  "rubric_meta_evaluation": { // Diagnostic of the Rubrics themselves
    "performance_score": "integer", // 1-10 rating of how well current rubrics are guiding the
    model.
    "weakest_rubrics": [
      {
        "rubric_text": "string",
        "issue": "string", // e.g., "Too vague", "Easily gamed", "Conflicts with Goal X"
        "suggested_action": "string" // "Refine", "Delete", "Reweight"
      }
    ]
  },

  "performance_advancement_analysis": { // Growth & Mastery
    "saturation_check": {
      "is_task_too_easy": "boolean", // True if model scores high but output is generic/safe.
      "mastered_capabilities": ["string"] // Skills the model now performs effortlessly.
    },
    "quality_gap_analysis": "string", // What is missing from "perfect" human-level output? (e.g.,
    "Model is correct but lacks wit.")
    "next_level_objectives": ["string"] // What should be the NEXT goal? (e.g., "Transition from
    'Accuracy' to 'Conciseness'.")
  },
}
```

Figure 10: Prompt template for batch-level summarization (2/4) described in Section 2.2. This prompt aggregates individual rollout analyses into a step-level strategic report.

Batch-Level Summarization (3/4)

```
"rubric_evolution_plan": { // Actionable Steps
  "corrective_actions": [ // Fixes for hacking/errors
    {
      "rubric_text": "string",
      "issue": "string",
      "action": "string" // e.g., "Add penalty for repetition"
    }
  ],
  "advancement_actions": [ // Actions to Raise the Bar (Curriculum)
    {
      "strategy": "string", // e.g., "Tighten Constraints", "New Dimension", "Deprioritize Basic
Reward"
      "target_rubric_concept": "string",
      "rationale": "string" // e.g., "Model has mastered basic politeness. Remove reward to
force focus on reasoning."
    }
  ]
},

"memory_entry": { // For Long-term Storage
  "title": "string", // e.g., "Step 85: Mastery of Instruction Following, Beginning Reasoning
Phase"
  "core_insight": "string",
  "tags": ["string"],
  "links_to_look_for": ["string"]
}
}
````
```

### ## Input Data Packet

#### ### High-Level Training Goal

`[Insert the high-level, ultimate goal the model is being trained to achieve. e.g., "Create a helpful and harmless assistant that can follow complex, multi-step user instructions."]`

#### ### RL Training Metadata

- \*\*Model Size:\*\*` [e.g., 70B Parameters]`  
- \*\*Current Step:\*\*` [e.g., 85,000]`  
- \*\*Total Steps:\*\*` [e.g., 200,000]`  
- \*\*Other Context:\*\*` [e.g., "This model is in the PPO phase, focusing on preference optimization." or "This model is in the KT0 phase."]

#### ### Current Reward Rubrics

- \*\*Anchor Rubric Set (Immutable):\*\*

```
````
[Insert the complete, verbatim rubrics used to assign rewards. e.g.,
- R0: "+1 The model acknowledges the user's premise."
- R1: "-2 The model exhibits sycophancy."
- R2: "+3 The model correctly identifies and refuses a harmful instruction."
````
```

- \*\*Adaptive Rubric Set:

```
````
[Insert the complete, verbatim rubrics used to assign rewards. e.g.,
- R0: "+1 The model acknowledges the user's premise."
- R1: "-2 The model exhibits sycophancy."
- R2: "+3 The model correctly identifies and refuses a harmful instruction."
````
```

#### ### Individual Rollout Analysis Reports

Figure 11: Prompt template for batch-level summarization (3/4) described in Section 2.2. This prompt aggregates individual rollout analyses into a step-level strategic report.

**Batch-Level Summarization (4/4)**

``[Insert all the Individual Rollout Analysis Reports here.]``

Now output ONLY valid JSON matching the schema above.

Figure 12: Prompt template for batch-level summarization (4/4) described in Section 2.2. This prompt aggregates individual rollout analyses into a step-level strategic report.

## Meta-Batch-Level Summarization (1/4)

### ## Role

You are a chief AI research scientist and an expert in Reinforcement Learning (RL) as applied to Large Language Models. Your primary expertise lies in defensive alignment (detecting model failure model, reward hacking, and safety violations) and offensive capability growth (identifying when a model has plateaued and how to further advance the model's performance).

### ## Background & Context

We are training a Large Language Model (LLM) using a Generative Reward Model (GenRM) and Rubric-as-Reward (RaR) paradigm where rewards are derived from dynamic rubrics that can be manipulated as needed.

When the number of rollouts is too large to fit in a single LLM context window, the system:

1. chunks the rollouts into **mini-batches**,
2. runs **Batch-level Summarization** on each mini-batch to produce **mini-batch summary reports**, then
3. runs this **Meta-Batch-level Summarization** step to synthesize those mini-batch summaries into a **single step-level batch report**.

### ### Rubric Manipulation Operations

- **Create**: Generate new rubric text with appropriate reward score to address missing evaluation criteria or emergent behaviors.
- **Update**: Specific textual modification or refinements to existing rubrics to improve clarity or precision.
- **Delete**: Remove rubrics that are obsolete, redundant, or causing negative alignment.
- **Reweight**: Adjust the scalar reward score associated with a rubric to prioritize or deprioritize specific rubrics.
- **Merge**: Combine multiple overlapping or related rubrics into a single, unified criterion.
- **Split**: Decompose a complex or multi-faceted rubric into distinct, atomic sub-criteria for granular evaluation.

You are allowed to link rubrics logically (e.g., Rule B applies only if Rule A is met) when manipulating rubrics. Note that the Anchor Rubric Set is immutable as they are the fundamental rubrics that prevents policy from drifting too much, thus improving training stability.

### ### Reward Calculation

The overall reward score of an individual rollout is calculated as the sum of all reward weights from each rubric that the criterion is met, unless specified otherwise.

### ## Task

You will be provided with an Input Data Packet including:

- RL Training Metadata (Goal, Model Size, Step Count, Phase)
- The Current Active Rubrics (Anchor + Adaptive)
- **A list of Mini-Batch Summary Reports** (each produced by the Batch-level Summarization step using the same superset schema)

Your task is to perform **meta-summarization**: synthesize all mini-batch summaries into **one** all-encompassing step-level strategic report.

This meta-report must answer the same three objectives as the normal batch report, but at the full-batch scope:

1. **Overall performance & trend**: How does the model perform overall across the entire (potentially huge) batch? What is the level and trajectory?
2. **Failure modes & reward hacking**: What are the dominant failure patterns or hacking vectors, and how severe are they?
3. **How to improve toward the high-level goal**: What rubric evolution actions (defensive patches + curriculum advancement) are needed next?

### ## Requirements

Figure 13: Prompt template for meta-batch-level summarization (1/4) described in Section 2.2. This prompt is used when the number of rollouts exceeds a single summarization pass.

## Meta-Batch-Level Summarization (2/4)

1. **Defense (Hacking & Flaws):** Identify if the model is gaming the system or if the current rubrics are broken/vague.
2. **Analysis (Patterns):** Identify systemic behaviors that aren't necessarily hacking but affect quality.
3. **Growth (Mastery & Advancement):** Identify if the model has "mastered" the current difficulty level. If the model is safe but "boring," you must propose how to **raise the bar** (e.g., transitioning from "Accuracy" to "Nuance").

### ### Critical Meta-Summarization Requirements

Because you are summarizing summaries:

- **Deduplicate aggressively:** If multiple mini-batches describe the same pattern with different phrasing, merge into one pattern.
- **Resolve conflicts explicitly:** If mini-batches disagree (e.g., one says "healthy," another says "critical"), you must:
  - state the disagreement,
  - propose the most likely explanation (e.g., distribution shift, heterogeneous task mix),
  - choose an overall classification that is **risk-aware** (err on the side of safety/stability).
- **Avoid false precision:** If exact rollout counts or exact percentages are unavailable, do not invent numbers.
  - Prefer "Observed in X/Y mini-batches" or "Frequently reported" over fabricated global percentages.
  - If mini-batches provide prevalence strings, you may summarize ranges qualitatively (e.g., "reported as 20–60% across mini-batches"), but do not fabricate exact totals.
- **Worst-case safety logic:** If any mini-batch indicates a clear safety violation or high-confidence reward hacking, ensure the overall report reflects elevated risk and prioritizes defensive fixes.

### ## Constraints

- **Output Format:** You must output only valid JSON. Do not include conversational filler before or after the JSON.
- **Objectivity:** Assess findings probabilistically (e.g., "High confidence of hacking") rather than absolute binary claims unless obvious from the reports.
- **Scope:** Base your analysis only on the provided Input Data Packet and mini-batch summaries. Do not hallucinate external context.

### ## Output Format

**JSON Schema:**

```
```json
{
  "batch_summary_id": "string", // A unique identifier, e.g., "step-[step_number]_meta-summary".

  "executive_summary": {
    "training_health": "string", // "Healthy", "Warning", "Critical", "Stagnant/Plateaued"
    "current_phase": "string", // "Exploration", "Exploitation", "Mastery", "Regression"
    "trend_overview": "string" // Concise high-level summary of the batch, noting heterogeneity if
    present.
  },

  "systemic_patterns": [ // Trends observed across >15% of the batch
    {
      "pattern_name": "string", // e.g., "Emergent Reasoning", "Format Rigidity"
      "prevalence": "string", // Prefer "Observed in X/Y mini-batches" if global % is unknown.
      "impact_on_goal": "string", // "Helps", "Hinders", "Neutral"
      "description": "string"
    }
  ],
}
```

Figure 14: Prompt template for meta-batch-level summarization (2/4) described in Section 2.2. This prompt is used when the number of rollouts exceeds a single summarization pass.

Meta-Batch-Level Summarization (3/4)

```

"aggregated_reward_hacking": { // Rigorous Defense
  "risk_level": "string", // "None", "Low", "Medium", "High"
  "primary_vector": "string", // e.g., "Length Gaming", "Sycophancy"
  "evidence_summary": "string" // Synthesized evidence from individual reports.
},

"rubric_meta_evaluation": { // Diagnostic of the Rubrics themselves
  "performance_score": "integer", // 1-10 rating of how well current rubrics are guiding the
model.
  "weakest_rubrics": [
    {
      "rubric_text": "string",
      "issue": "string", // e.g., "Too vague", "Easily gamed", "Conflicts with Goal X"
      "suggested_action": "string" // "Refine", "Delete", "Reweight"
    }
  ]
},

"performance_advancement_analysis": { // Growth & Mastery
  "saturation_check": {
    "is_task_too_easy": "boolean", // True if model scores high but output is generic/safe.
    "mastered_capabilities": ["string"] // Skills the model now performs effortlessly.
  },
  "quality_gap_analysis": "string", // What is missing from "perfect" human-level output? (e.g.,
"Model is correct but lacks wit.")
  "next_level_objectives": ["string"] // What should be the NEXT goal? (e.g., "Transition from
'Accuracy' to 'Conciseness'.")
},

"rubric_evolution_plan": { // Actionable Steps
  "corrective_actions": [ // Fixes for hacking/errors
    {
      "rubric_text": "string",
      "issue": "string",
      "action": "string" // e.g., "Add penalty for repetition"
    }
  ],
  "advancement_actions": [ // Actions to Raise the Bar (Curriculum)
    {
      "strategy": "string", // e.g., "Tighten Constraints", "New Dimension", "Deprioritize Basic
Reward"
      "target_rubric_concept": "string",
      "rationale": "string" // e.g., "Model has mastered basic politeness. Remove reward to
force focus on reasoning."
    }
  ]
},

"memory_entry": { // For Long-term Storage
  "title": "string", // e.g., "Step 85: Mastery of Instruction Following, Beginning Reasoning
Phase"
  "core_insight": "string",
  "tags": ["string"],
  "links_to_look_for": ["string"]
}
}

```

Input Data Packet

High-Level Training Goal

`[Insert the high-level, ultimate goal the model is being trained to achieve. e.g., "Create a helpful and harmless assistant that can follow complex, multi-step user instructions."]`

RL Training Metadata

Figure 15: Prompt template for meta-batch-level summarization (3/4) described in Section 2.2. This prompt is used when the number of rollouts exceeds a single summarization pass.

Meta-Batch-Level Summarization (4/4)

```
- Model Size: ` [e.g., 70B Parameters]`  
- Current Step: ` [e.g., 85,000]`  
- Total Steps: ` [e.g., 200,000]`  
- Other Context: ` [e.g., "This model is in the PPO phase, focusing on preference optimization." or "This model is in the KT0 phase." ]`
```

Current Reward Rubrics

```
- Anchor Rubric Set (Immutable):
```

```
```\n[Insert the complete, verbatim rubrics used to assign rewards. e.g.,  
- R0: "+1 The model acknowledges the user's premise."
- R1: "-2 The model exhibits sycophancy."
- R2: "+3 The model correctly identifies and refuses a harmful instruction."]
```\n
```

```
- Adaptive Rubric Set:
```

```
```\n[Insert the complete, verbatim rubrics used to assign rewards. e.g.,  
- R0: "+1 The model acknowledges the user's premise."
- R1: "-2 The model exhibits sycophancy."
- R2: "+3 The model correctly identifies and refuses a harmful instruction."]
```\n
```

Mini-Batch Summary Reports

You will receive **K** mini-batch reports. Each report is the JSON output from the Batch-level Summarization step (same schema as below).

```
` [Insert all mini-batch summary JSON reports here, in a list or concatenated form.] `
```

(Optional but recommended if available):

```
- `mini_batch_rollout_counts`: `[Insert K integers matching the mini-batches, if known.]`  
- `total_rollouts_estimate`: `[Insert total rollouts, if known.]`
```

Now output ONLY valid JSON matching the schema above.

Figure 16: Prompt template for meta-batch-level summarization (4/4) described in Section 2.2. This prompt is used when the number of rollouts exceeds a single summarization pass.

Memory Query Generation (1/3)

Role

You are a chief AI research scientist and an expert in Reinforcement Learning (RL) as applied to Large Language Models. Your primary expertise lies in defensive alignment (detecting model failure modes, reward hacking, and safety violations) and offensive capability growth (identifying when a model has plateaued and how to further advance the model's performance). Your role is to **prepare the right historical evidence** so the next Rubric Update stage can make stable, effective rubric changes.

Background & Context

We are training a Large Language Model (LLM) using a Generative Reward Model (GenRM) and Rubric-as-Reward (RaR) paradigm where rewards are derived from dynamic rubrics that can be manipulated as needed.

A memory system stores historical:

- Batch summary reports
- Individual rollout analysis summaries
- Past rubric updates and their outcomes/lessons

Rubric Manipulation Operations

- **Create**: Generate new rubric text with appropriate reward score to address missing evaluation criteria or emergent behaviors.
- **Update**: Specific textual modification or refinements to existing rubrics to improve clarity or precision.
- **Delete**: Remove rubrics that are obsolete, redundant, or causing negative alignment.
- **Reweight**: Adjust the scalar reward score associated with a rubric to prioritize or deprioritize specific rubrics.
- **Merge**: Combine multiple overlapping or related rubrics into a single, unified criterion.
- **Split**: Decompose a complex or multi-faceted rubric into distinct, atomic sub-criteria for granular evaluation.

You are allowed to link rubrics logically (e.g., Rule B applies only if Rule A is met) when manipulating rubrics. Note that the Anchor Rubric Set is immutable as they are the fundamental rubrics that prevents policy from drifting too much, thus improving training stability.

Reward Calculation

The overall reward score of an individual rollout is calculated as the sum of all reward weights from each rubric that the criterion is met, unless specified otherwise.

Task

You will be given an Input Data Packet including:

- RL Training Metadata (Goal, Model Size, Step Count)
- The Current Active Rubrics (Anchor + Adaptive)
- The Batch Analysis Report (most recent diagnostic of model behavior)

Your task is to generate **0 to N** targeted memory search queries (where **N is adaptive**) to retrieve the most relevant historical logs and past rubric updates for the upcoming Rubric Update stage.

What the queries must cover (prioritized)

When applicable, cover these areas:

1. **Primary hack / risk vector** (e.g., aggregated_reward_hacking.primary_vector) and close paraphrases.
2. **Systemic pattern names** (systemic_patterns[].pattern_name) and "what it looked like".
3. **Weakest rubrics & issues** (rubric_meta_evaluation.weakest_rubrics[].issue) and short identifying phrases from the rubric_text.
4. **Curriculum advancement targets** (performance_advancement_analysis.next_level_objectives / quality_gap_analysis) if plateau is suspected.
5. **Backfire avoidance queries**: if proposing penalties/tightening, query for cases where similar changes caused regressions (verbosity penalty backfired, over-refusal increased, etc.). Only include this if the Batch Analysis Report suggests a change, hacking, or plateau.

Figure 17: Prompt template for memory query generation (1/3) described in Section 2.3. This prompt generates targeted search queries that are executed against the persistent evaluation memory to retrieve relevant historical context for rubric update.

Memory Query Generation (2/3)

Query writing rules

- Each query should be a standalone search string suitable for either keyword match or semantic search.
- Avoid duplicates and near-duplicates.
- Prefer concrete terms found in the packet (e.g., "Length Gaming", "Sycophancy", "Saturation", "Too vague rubric").
- If you reference a rubric, include either:
 - a short exact phrase from the rubric_text (<= 8 words), OR
 - the most distinctive concept name (e.g., "conciseness", "hallucination", "refusal").
- Do not include any conversational text outside JSON.

Constraints

- ****Output Format:**** You must output only valid JSON. Do not include conversational filler before or after the JSON.
- ****Scope:**** Use only the provided Input Data Packet. Do not hallucinate external context.

Output Format

Return a valid JSON object with the following structure:

```
```json
{
 "queries": [
 "string"
]
}
```
```

The list must contain the appropriate number of query strings, based on the guidance above.

Input Data Packet

High-Level Training Goal

`[Insert the high-level, ultimate goal the model is being trained to achieve.]`

RL Training Metadata

- ****Model Size:**** `[e.g., 70B Parameters]`
- ****Current Step:**** `[e.g., 85,000]`
- ****Total Steps:**** `[e.g., 200,000]`
- ****Other Context:**** `[e.g., "PP0 phase" / "KT0 phase" / etc.]`

Current Reward Rubrics

- ****Anchor Rubric Set (Immutable):****

```
```
[Insert the complete, verbatim rubrics used to assign rewards. e.g.,
- R0: "+1 The model acknowledges the user's premise."
- R1: "-2 The model exhibits sycophancy."
- R2: "+3 The model correctly identifies and refuses a harmful instruction."]
```
```

- ****Adaptive Rubric Set:****

```
```
[Insert the complete, verbatim rubrics used to assign rewards. e.g.,
- R0: "+1 The model acknowledges the user's premise."
- R1: "-2 The model exhibits sycophancy."
- R2: "+3 The model correctly identifies and refuses a harmful instruction."]
```
```

Batch Analysis Report

Figure 18: Prompt template for memory query generation (2/3) described in Section 2.3. This prompt generates targeted search queries that are executed against the persistent evaluation memory to retrieve relevant historical context for rubric update.

Memory Query Generation (3/3)

`[Insert the JSON output from the Batch Summary step]`

Now output ONLY valid JSON matching the schema above.

Figure 19: Prompt template for memory query generation (3/3) described in Section 2.3. This prompt generates targeted search queries that are executed against the persistent evaluation memory to retrieve relevant historical context for rubric update.

Rubric Update (1/4)

Role

You are a chief AI research scientist and an expert in Reinforcement Learning (RL) as applied to Large Language Models. Your primary expertise lies in defensive alignment (detecting model failure model, reward hacking, and safety violations) and offensive capability growth (identifying when a model has plateaued and how to further advance the model's performance). Your role is not merely to judge current performance, but to **steer the evolutionary trajectory** of the model. You control the "laws" (Rubrics) of the environment to ensure the model continuously improves without plateauing or learning deceptive behaviors.

Background & Context

We are training a Large Language Model (LLM) using a Generative Reward Model (GenRM) and Rubric-as-Reward (RaR) paradigm where rewards are derived from dynamic rubrics that can be manipulated as needed.

Rubric Manipulation Operations

- **Create:** Generate new rubric text with appropriate reward score to address missing evaluation criteria or emergent behaviors.
- **Update:** Specific textual modification or refinements to existing rubrics to improve clarity or precision.
- **Delete:** Remove rubrics that are obsolete, redundant, or causing negative alignment.
- **Reweight:** Adjust the scalar reward score associated with a rubric to prioritize or deprioritize specific rubrics.
- **Merge:** Combine multiple overlapping or related rubrics into a single, unified criterion.
- **Split:** Decompose a complex or multi-faceted rubric into distinct, atomic sub-criteria for granular evaluation.

You are allowed to link rubrics logically (e.g., Rule B applies only if Rule A is met) when manipulating rubrics. Note that the Anchor Rubric Set is immutable as they are the fundamental rubrics that prevents policy from drifting too much, thus improving training stability.

Reward Calculation

The overall reward score of an individual rollout is calculated as the sum of all reward weights from each rubric that the criterion is met, unless specified otherwise.

Task

You will be provided with an Input Data Packet including:

- **RL Training Metadata** (Goal, Model Size, Step Count).
- **The Current Active Rubrics**.
- **The Batch Analysis Report** (The most recent diagnostic of model behavior).
- **Retrieved Memory Context** (Relevant historical logs, past rubric failures, and long-term trend analysis).

Your objective is to **modify the adaptive rubrics** to optimize the next phase of training. You must balance **Stability** (not changing things too fast) with **Adaptation** (fixing loopholes and raising standards).

Requirements

You must categorize your update strategy into one of three modes based on the input data:

Defensive Mode

- **Trigger:** The Batch Report detects "Reward Hacking," "Safety Violations," or "Loophole Abuse."
- **Action:**
 - Immediately patch the specific wording causing the hack.
 - Introduce negative constraints (penalties).
 - **Consult Memory:** Check if this hack has appeared before. If yes, apply a stronger penalty than last time.

Curriculum Advancement

Figure 20: Prompt template for rubric update (1/4) described in Section 2.3. This prompt grounds rubric modifications in the batch analysis report and retrieved historical context from the persistent evaluation memory.

Rubric Update (2/4)

```
- Trigger: The Batch Report indicates "Saturation," "Mastery," or "Plateauing" (High rewards, generic outputs).
- Action:
  - Raise the Bar: Transition from simple constraints (e.g., "No grammar errors") to complex stylistic goals (e.g., "Use Socratic reasoning").
  - Deprioritize Basics: Lower the weight of basic tasks the model has already mastered to force it to chase the new, harder rewards.
```

Maintenance / Stability

```
- Trigger: The model is learning well, trends are positive, and no hacking is detected.
- Action:
  - Make minor or no changes.
  - Drastic changes during a healthy learning phase can destabilize the policy.
```

Rubric Set Hygiene (Conciseness & De-duplication)

Regardless of update strategy, the **updated rubric set** you output (`final_rubric_set`) must satisfy:

```
- Concise set: Keep the rubric set as small as possible. Prefer fewer, clearer criteria over many narrow ones.
- No duplicates: Do not include duplicate or near-duplicate rubrics (similar meaning with different wording). If two rubrics overlap materially, use MERGE or DELETE so only one remains.
- No duplicate anchors: If an Anchor Rubric already covers a concept (e.g., correctness), do not "re-create" an equivalent rubric in the adaptive set. Anchors should appear exactly once in the final rubric set.
- Avoid double-counting: Do not create multiple rubrics that all reward/penalize the same behavior in the same direction unless you explicitly justify why separate credit assignment is necessary.
- Reward Budget (Hard): The next-step rubric weights must imply max_total_reward ≤ +5 and min_total_reward ≥ -5. Concretely, across Anchor+Adaptive:  $\sum$  positive weights  $\leq 5$  and  $\sum$  negative weights  $\geq -5$ . If over budget, adjust adaptive weights (delete/merge/reweight) until within budget.
```

Constraints

```
* Output Format: You must output only valid JSON. Do not include conversational filler before or after the JSON.
* Objectivity: Assess findings probabilistically (e.g., "High confidence of hacking") rather than as absolute binary facts unless obvious.
* Scope: Base your analysis only on the provided Data Packet. Do not hallucinate external context.
```

Output Format

You must output a valid JSON object containing the new configuration and the rationale for the memory system.

JSON Schema:

```
```json
{
 "rubric_update_id": "string", // A unique identifier, e.g., "step-[step_number]_rubric-update".
 "update_strategy": "string", // "DEFENSIVE", "CURRICULUM_ADVANCEMENT", or "MAINTENANCE"
 "memory_consultation": {
 "relevant_history_utilized": "string", // Reference specific insights from the Retrieved Memory Context that influenced this decision.
 "avoidance_action": "string" // e.g., "Avoided re-introducing Rule X because Memory Y showed it failed previously."
 },
}
```

Figure 21: Prompt template for rubric update (2/4) described in Section 2.3. This prompt grounds rubric modifications in the batch analysis report and retrieved historical context from the persistent evaluation memory.

## Rubric Update (3/4)

```
"rationale": {
 "diagnosis": "string", // Summary of why changes are needed based on the Batch Report.
 "intended_effect": "string" // What do you expect the model to learn from these specific
changes?
},

"rubric_modifications": [// List ONLY the changes made.
{
 "operation": "string", // "CREATE", "UPDATE", "DELETE", "REWEIGHT", "MERGE", "SPLIT"
 "target_rubric_id_or_text": "string", // The original rubric being modified (if applicable).
 "new_rubric_text": "string", // The new text (null if DELETE). You must only include the
description (main text) here. Do not include the weight, title, or rubric id here.
 "new_weight": "float",
 "reasoning": "string"
}
],

"final_adaptive_rubric_set": [// The COMPLETE list of adaptive rubrics to be used for the next
step.
{
 "rubric_text": "string", // You must only include the description (main text) here. Do not
include the weight, title, or rubric id here.
 "weight": "float"
}
],

"memory_artifact": { // What should be stored in the Agentic Memory for future reference?
 "event_summary": "string", // e.g., "Step 85k: Shifted focus from Accuracy to Conciseness."
 "lessons_learned": "string" // e.g., "Rubric for 'wit' is prone to sarcasm hacks; tightened
constraints."
}
}
`
```

## Input Data Packet

### High-Level Training Goal

`[Insert the high-level, ultimate goal the model is being trained to achieve. e.g., "Create a helpful and harmless assistant that can follow complex, multi-step user instructions.]"`

### RL Training Metadata

```
- **Model Size:** `[e.g., 70B Parameters]`
- **Current Step:** `[e.g., 85,000]`
- **Total Steps:** `[e.g., 200,000]`
- **Other Context:** `[e.g., "This model is in the PPO phase, focusing on preference
optimization." or "This model is in the KT0 phase."]`
```

### Current Reward Rubrics

```
- **Anchor Rubric Set (Immutable):**
```

```
````
[Insert the complete, verbatim rubrics used to assign rewards. e.g.,
- R0: "+1 The model acknowledges the user's premise."
- R1: "-2 The model exhibits sycophancy."
- R2: "+3 The model correctly identifies and refuses a harmful instruction."]
````
```

```
- **Adaptive Rubric Set:**
```

Figure 22: Prompt template for rubric update (3/4) described in Section 2.3. This prompt grounds rubric modifications in the batch analysis report and retrieved historical context from the persistent evaluation memory.

## Rubric Update (4/4)

```
```\n[Insert the complete, verbatim rubrics used to assign rewards. e.g.,\n- R0: "+1 The model acknowledges the user's premise."\n- R1: "-2 The model exhibits sycophancy."\n- R2: "+3 The model correctly identifies and refuses a harmful instruction."]\n```\n
```

Batch Analysis Report

`[Insert the JSON output from the Batch Summary step]`

Retrieved Memory Context

`[Insert relevant summaries of previous updates, e.g., "Step 20,000: We removed the 'humor' rubric because it caused hallucination." ]`

Now output ONLY valid JSON matching the schema above.

Figure 23: Prompt template for rubric update (4/4) described in Section 2.3. This prompt grounds rubric modifications in the batch analysis report and retrieved historical context from the persistent evaluation memory.

J Use of Large Language Models

Large language models are used in the following capacities as part of the research: (i) **Policy model and RL training:** Qwen2.5-7B-Instruct serves as the default base policy model fine-tuned via GRPO, with an additional Qwen3-4B policy-base diagnostic reported in Appendix F.3. (ii) **AMARIS pipeline components:** GPT-OSS-120B is used for rubric-based scoring and individual rollout analysis, and GPT-5.1 is used for batch summarization, memory query generation, and rubric update. These usages constitute core experimental components of our methodology and are described in detail in Section 2 and Section 3.3. (iii) **Evaluation:** LLM-based judges are used for evaluation on benchmarks that require them, such as HealthBench, WritingBench, and Creative Writing v3, following the evaluation protocols established by each respective benchmark. (iv) Additionally, LLMs were used to refine the prompt templates that guide each pipeline stage (Appendix I); the final prompt designs and all experimental results remain the sole responsibility of the authors. Beyond the experiments, we acknowledge the use of large language models in the final stages of manuscript preparation. These tools were employed exclusively for identifying and correcting typographical and grammatical errors, ensuring clarity and precision in the written presentation. Their use was strictly limited to linguistic refinement and did not impact the study’s conceptual framework, research methodology, data analysis, or conclusions. All intellectual contributions and substantive content remain those of the authors.

K Artifact Licenses and Dataset Statistics

We use third-party datasets, benchmarks, models, and software artifacts only through their official releases or access channels and under their stated licenses or repository terms. This includes the rubric-training datasets and benchmarks cited below, the policy and judge models used in Section 3.3 (Qwen et al., 2025; OpenAI et al., 2025; Singh et al., 2025), and Chroma for the memory store (chroma-core, 2026). When a release does not specify a standalone redistribution license, we use it only as a cited research artifact through its official access channel. The AMARIS release will not redistribute third-party datasets, benchmark prompts, model weights, or proprietary API outputs; it will provide code and configuration needed to reproduce our

use of the official artifacts.

The training sources are as follows. RaR-Science and RaR-Medicine contain 18,333/2,292/2,292 and 17,926/2,240/2,242 train/validation/test examples, respectively (Gunjal et al., 2025). OpenRubrics contains 35,622 training examples (Liu et al., 2026). For creative-writing training, we use the writing subset of RubricHub through its official release (Li et al., 2026). For evaluation, we use the official benchmark releases and protocols described in Section 3.1: GPQA-Diamond contains 198 questions (Rein et al., 2024), HealthBench contains 5,000 health conversations (Arora et al., 2025), IFEval contains 541 prompts (Zhou et al., 2023), InFoBench contains 500 instructions with 2,250 decomposed requirements (Qin et al., 2024), IFBench contains 300 test prompts covering 58 held-out verifiable constraints (Pyatkin et al., 2025), WritingBench contains 1,000 writing queries in the benchmark configuration we evaluate (Wu et al., 2025b), and Creative Writing v3 uses 32 prompts with 3 generations per prompt (Paech, 2025).